Guava: software evolution and architecture recovery

A study of 30 stable releases, from 10.0 to 33.7.2

Abstract

This study examines how Guava's internal class structure changes over time and how two clustering methods recover groups of related classes. Thirty releases are sampled across the project's history. The instructor's DependencyExtractor supplies the class dependencies; external libraries are excluded, and inner and anonymous classes are merged with their enclosing classes. ACDC and k-means are each run before and after removing classes identified by the supplied paper's Usage Equality method.

The sampled graph grows from 317 to 594 classes and from 1,289 to 2,750 dependencies. Removing widely used classes affects the recovery methods differently: on the same retained graph, the lecture's modularization-quality score improves for k-means in 29 of 30 releases and decreases for ACDC in 30 of 30. These results describe the extracted structure; they do not establish Guava's intended architecture. The report explains the data, parameter choices and evaluation, and includes every final measurement. [D1; D3]

Release dates: 2011-09-28 to 2026-09-29.
Dataset snapshot: 2026-10-17.
Project: github.com/google/guava.

Contents

1. Purpose and main findings . . . 3

2. The releases and their dependency graphs . . . 4

3. How the architectures are recovered and evaluated . . . 7

4. How Guava changes over time . . . 12

5. Comparing the recovered architectures . . . 16

6. The AI-assisted architecture proposal . . . 23

7. Discussion and practical limits . . . 28

8. Reproducing the analysis . . . 30

Appendix A. The 30 selected releases . . . 32

Appendix B. All final quality measurements . . . 33

References . . . 37

How to read the report

Section 1 gives the purpose and main findings. Sections 2 and 3 explain how the data was obtained and how the experiments work. Sections 4 and 5 present the evolution and clustering results; Sections 6 and 7 discuss the AI proposal and what the results mean.

The reproduction commands are in Section 8. The appendices contain every selected release and all 120 final quality measurements. Citations such as [L67] refer to the supplied lectures; [D1] and similar labels point to saved repository evidence. The reference list gives their full names and paths.

1. Purpose and main findings

1.1 Introduction

This study follows Guava through 30 releases to examine how its structure changes and how that structure can be divided into modules. It asks how class and dependency counts develop over time, and whether removing widely used classes helps clustering algorithms recover more cohesive groups. Inner and anonymous classes are merged with their enclosing top-level class, and external libraries are excluded. A dependency is a distinct directed relation between two classes in this inventory. [D1; D4; D6]

Each release is analysed with ACDC and k-means, before and after applying the paper's Usage Equality method. This method identifies broadly used classes whose connections may obscure subsystem boundaries. Filtering excludes their nodes and incident edges from the analysis graph; it does not change Guava's source code. The four conditions are labelled A1 to A4 below. [UE; AS pp.2–4]

Condition	Algorithm	Graph evaluated
A1	ACDC (+so, -l20)	All merged project classes
A2	Same ACDC configuration	Graph after Usage Equality filtering
A3	k-means on normalized dependency features	All merged project classes
A4	Same features and release-specific k	Graph after Usage Equality filtering

A pilot on one release sets the parameters. Each release then has one final partition for each condition, keeping the four required comparisons without repeating parameter experiments across all versions. The ten initializations inside a k-means fit produce a single final partition. A separate AI proposal is evaluated for the latest release. [AS pp.2–4]

Main findings

Between the first and last samples, the class count rises by 87.4% and the dependency count by 113.3%. Usage Equality selects 6–23 classes in a release, with 32 distinct class names selected at least once. The gaps between samples mean that these trends cannot identify the exact commit responsible for a change. [D1; D2; L12 pp.35–48]

Modularization quality (MQ) combines average cohesion within groups and coupling between groups. To compare the effect of regrouping, both partitions are scored on the same remaining classes and edges. The table counts the releases in which MQ changes. [D3; D9]

Method	Higher MQ	Lower MQ	Unchanged
ACDC	0	30	0
k-means	29	1	0

Changes within 10^{-12} count as unchanged. Higher MQ describes the extracted structure; it does not establish the architecture intended by Guava's developers. [D3; D9]

The AI proposal places the latest 594 classes into 18 named modules. Its MQ is 0.0943, compared with 0.1790 for k-means and 0.2553 for ACDC on the full graph. Because the AI also uses names and package documentation, this is not a comparison of methods given identical information. [D12]

2. The releases and their dependency graphs

2.1 Guava and the release sample

The subject is **Guava**, the Java library maintained in the google/guava repository. The analysis follows the classes shipped in its main production JAR across 30 stable releases. Using this artifact throughout gives the study a consistent scope: it describes the library's class architecture, while other modules in the repository fall outside the sample. [G1; D4]

Course eligibility criterion	Recorded evidence
Java; latest release ≥10,000 LOC	GitHub primary language: Java. 96,622 nonblank, noncomment lines in owned Java sources.
GitHub repository created before 2024	2014-05-29
Not a fork; not archived	fork=false; archived=false at the recorded snapshot.
≥50 commits since 2024-01-01	100 distinct qualifying commits saved as evidence that the minimum is met; this is not a count of all commits.
≥10 major/minor releases	30 stable releases covering exactly 30 distinct major/minor families.

The line count used for course eligibility includes Guava-owned package descriptors. The source files represented by class nodes in the dependency graph contain 96,546 code lines at the latest release. Both counts exceed the requirement, but they describe slightly different sets of files. [D4; D6]

How the 30 releases were chosen

The selection rule was set before examining the quality scores. Of the 124 tag references in the saved catalog, 82 match the numeric stable-release pattern. The other 42 include prereleases, auxiliary artifacts and alternative names, so they should not all be described as beta releases. Excluding nine numeric releases before 10.0, the ambiguous duplicate naming of 13.0, and two later backports to earlier release lines leaves 70 mainline candidates in chronological order. [D4]

The sample starts with the first eligible .0 release from each of the 24 major versions 10 through 33. Version 13.0.1 represents major 13 because the ambiguous 13.0 tags are excluded. The latest release, 33.7.2, supplies the final endpoint. This leaves five places to fill with minor releases from the history. Each major/minor family contributes its first eligible release, except for the explicit latest endpoint. [D4]

> Calendar-gap reduction from adding a release:
> $(L + R)^2 - L^2 - R^2 = \mathbf{2LR}$

For each remaining place, the selector chooses the candidate that most reduces the sum of squared time gaps. L and R are the days from that candidate to its nearest selected releases on either side. Every resulting interval must span at least 60 days, and each major version may receive at most two of these five optional additions. Ties choose the earlier date, then the lower numeric version. The rule uses release dates only; it never reads class counts, dependency graphs or quality scores. [D4]

The resulting 30 observations are between 74 and 471 days apart, with a median gap of 141 days. All 24 major versions are represented and no major/minor family repeats. The five additions are 27.1, 28.2, 31.1,

33.4.0 and 33.5.0. Although 33.4 and 33.5 are numerically adjacent minor versions, their publication dates are 275 days apart. Appendix A lists the complete sample. [D4]

The selection does not require dependency graphs to differ: an unchanged graph remains a valid observation. Plain artifacts precede 23.1; the JRE artifact is used after the variant split. Android variants do not add observations. [D4]

Dates for older releases come from Guava's release-history pages; later dates use GitHub Release publication timestamps. Git commit dates are stored separately. The repository evidence establishes the eligibility criteria above; instructor approval is not recorded in the available material. [AS p.1; G2; D4]

2.2 Where the data came from

For each release, the dataset records the Git commit behind its tag and downloads three files from Maven Central: the production binary JAR, the source JAR and the Project Object Model (POM), which identifies the published artifact. The extractor reads the published bytecode. This avoids having to rebuild every historical version and then dropping releases whose old build tools no longer work. Downloading the data does not execute Guava code. [D4; D5]

Evidence layer	Stored and checked
Release identity	Git tag name, 40-character commit SHA, publication date, the source of that date and Maven artifact coordinate.
Artifact integrity	Pinned SHA-256 for every downloaded file; Maven SHA-1 sidecars; ZIP CRC checks; POM version/artifact identity.
Source ownership	Per-source-file path and checksum, source-owned class inventory, SLOC definition and excluded binary entries.
Source-to-Git audit	Every Java source-JAR file at the oldest and newest selected endpoints compared byte for byte with the pinned Git archive.
Offline reproducibility	Cached files must match the saved checksums. A corrupt file causes a failure rather than an automatic replacement.

Endpoint	Java files	Git/source result	Commit prefix
10.0	326	All byte-identical	baf008c5a3de
33.7.2	611	All byte-identical	666338cc49d5

All **90 files** (116.25 MiB) matched their saved checksums and passed the checks listed above. At the first and last releases, every Java file in the source JAR was also compared byte for byte with the pinned Git archive. The intermediate 28 releases have saved artifact checksums and Git identities, but did not receive that complete per-file comparison. SHA sidecars downloaded from the same repository help detect corruption; they do not independently authenticate the publisher. PGP signatures were not verified. [D5]

Tracing a result back to its source

A result can be explained in the oral exam by following one release through the saved files. Its row in `config/versions.json` gives the tag SHA and Maven URL. The downloaded JAR can be hashed and checked against that record. The source manifest then shows which classes belong to Guava, and the extractor records show how those classes become the class and dependency CSVs used for clustering. This connects a reported measurement to the particular upstream file from which it was obtained. [L4 pp.11–15]

The supporting files are config/versions.json; data/provenance/SHA256SUMS; acquisition.json; source_inventory_<version>.json; git_source_match_10.0.json; git_source_match_33.7.2.json. All paths are repository-relative.

2.3 From Java classes to a dependency graph

A node represents a Guava-owned type; A→B means A depends on B. The instructor's unmodified DependencyExtractor supplies every primary edge. A separate ASM bytecode pass identifies owned classes, links nested classes to their enclosing types and resolves the member references used by Usage Equality. Its extra annotation, type and declaring-owner relationships remain outside the primary graph. [TUT p.6; D6]

The instructor receives a deterministic JAR of owned base classes, copied unchanged from the Maven binary. Per-entry checksums verify the copies, and the original JAR is preserved. Its original CompCalc.MainClass runs with an explicit classpath and unchanged bundled libraries. [D6]

Evidence	Role
classes.csv; class_aliases.csv	Owned common/publicsuffix types, with genuine nested, local and anonymous classes merged using enclosing-class metadata.
instructor_raw.csv	Original seven-category output, including occurrence counts retained for auditing.
dependencies_typed.csv; dependencies.csv	Instructor relationships reduced first to unique typed pairs, then to unique directed class pairs; self-loops and external endpoints removed.
member_dependencies.csv	Resolved method and field references for Usage Equality. Binary member identities remain distinct when their owning classes are merged.
supplemental_dependencies.csv	The broader graph from ASM, saved separately to examine extraction differences.

All seven tutorial relations are selected: EXTENDS, IMPLEMENTS, METHOD_CALL, METHOD_PARAMETER, FIELD_REFERENCE, TYPE_USAGE and EXCEPTION_THROWS. Repeated occurrences and relation types become one unweighted edge per ordered class pair. Coverage is limited to constructs recognized by the supplied executable. [TUT p.6]

Class processing	10.0	33.7.2
Owned binary classes before merging	1,157	1,948
Nested/anonymous identities merged	840	1,354
Canonical project-owned types	317	594
Owned annotation types	6	31
Primary-graph isolates	15	45

Ownership includes com.google.common and the com.google.thirdparty.publicsuffix sources. Their Guava Authors headers establish ownership despite the package name. JDK classes, failureaccess and all external libraries are excluded from both ends of every edge. [D4; D6]

Owned annotation types remain nodes, including isolates; annotation applications add no primary edges. An @Beta API belongs to its containing stable release. Module/package descriptors and multi-release overrides are excluded. This static model omits arbitrary reflection, erased constants and full virtual-dispatch expansion. [D6]

At the same ownership scope for 33.7.2, the instructor graph has 2,750 edges; the supplemental graph has 3,982. They share 2,750, with 1,232 supplemental-only edges, including 1,145 targeting annotations. Only instructor edges enter clustering and MQ. The saved audit explains these extraction differences. [D6]

3. How the architectures are recovered and evaluated

3.1 Identifying widely used classes with Usage Equality

A class with many callers can connect otherwise separate parts of the system and affect the recovered groups. However, many callers alone do not establish that the class is an omnipresent utility. Yano and Matsuo's Usage Equality (UE) method also asks whether any one caller has a particularly strong relationship with the class's members. The replacement tool, JNode-UE, implements this method from the supplied 2020 paper. [UE pp.663–664; D2]

This replacement uses UE on its own. The older JNode/Noise method described in the lectures uses ComponentRank and shortest-path significance; those rules are not part of this implementation. The distinction matters when comparing removal sets produced by the two methods. [L67 pp.46–47; D2]

For a caller class A and a target class B, $D_M(A,B)$ measures the caller's dedication to B through the members it uses. Usage equality e(B) is high when no caller has a large dedication score. Cardinality c(B) increases with the number of caller classes. Both thresholds must be met before B is removed. [UE pp.663–664]

$$D_M(A,B) = \Sigma_{m \in M(A,B)} \, 1 \,/\, [\text{xm_fanin}(m) \times \text{mx}(B)]$$

$$e(B) = 1 - \max_A D_M(A,B)$$

$$c(B) = 1 - 1 \,/\, \text{fanin}(B)$$

Remove B exactly when **e(B) ≥ 0.85** and **c(B) ≥ 0.90**.

Term	Precise meaning after class-owner normalization
M(A,B)	Distinct members of B referenced by members belonging to caller class A.
xm_fanin(m)	Distinct source members outside B that reference target member m; not caller classes.
mx(B)	Number of B's distinct members referenced from outside B; unreferenced members do not enter the denominator.
fanin(B)	Number of distinct other internal project classes with resolved member use of B.

With c(B) ≥ 0.90, a class needs at least ten caller classes to qualify. A class with no incoming member references has undefined scores and is retained. The calculation runs once on the original data, and all qualifying classes are removed together. The thresholds stay the same for every release; removal is not repeated to create a cascade. In these definitions, 'outside B' means another Guava-owned class. External libraries never enter the counts. The output records every score, the strongest caller and the removal decision. [UE; D2]

Why the number of callers is not enough

Consider a utility with one operation, used by one member in each of ten caller classes. Each caller's dedication is 1/10, giving e = c = 0.90, so the utility qualifies for removal. Now consider a logger with one shared operation used by 21 callers. A startup class also uses a separate initialization operation that no other class uses. Its dedication is 1/(21×2) + 1/(1×2), so e is approximately 0.476. The logger is retained because of this stronger relationship, despite its many callers. This example illustrates the calculation; it is not a measurement from Guava. [UE Fig.1; D2]

The implementation groups distinct member references with sets and dictionaries and caches the class that declares each member. It uses stable summation and an exact rational calculation when a score lies at a threshold. The expected cost of the main aggregation is O(R + C + N), followed by sorting for deterministic output: R is the number of unique member references, C the number of caller-target class pairs and N the number of classes. The method does not require all-pairs shortest paths. [D2]

3.2 How the architectures are recovered

ACDC

ACDC groups classes by looking for structural patterns in their dependencies. The study compiles the archived York Java implementation and runs it with `+so -l20`: SubGraph Dominator runs first, followed by Orphan Adoption to assign remaining classes. The BodyHeader pattern matches C source and header pairs, so it provides no evidence for this Java class graph. The result uses ACDC's default fine, flat DownInducer output. [TUT pp.7–8; L67 pp.19–24; ACDC; D7]

The value 20 limits the SubGraph construction stage. It does not cap the size of every final cluster: remaining orphans can end up in a larger container. Such clusters are reported as ACDC produces them. Classes with no dependencies, called isolates, need separate handling because the dependency-only RSF input cannot represent them. They are restored as singleton clusters. Every other class must occur exactly once in the returned partition; invalid, duplicate, missing or stale output causes a failure. [D7]

K-means

K-means groups classes with similar dependency patterns into k clusters. To make those patterns comparable across releases, each class is described using the same incoming and outgoing dependency coordinates. A is the adjacency matrix and I supplies the identity entries; the combined rows are normalized to unit length. [D7]

$$
\begin{aligned}
&U = \text{sorted union of class identities across all 30 releases} \\
&X = \text{row-L2-normalize}([A + I, (A + I)^{T}]) \\
&k_{full} = \max(1, \operatorname{ceil}(n_{full} \times k_{pilot} / n_{pilot})) \\
&k_{filtered} = \min(k_{full}, n_{filtered})
\end{aligned}
$$

There are 715 distinct class identities across the 30 releases, giving **1,430 columns** after including both dependency directions. A current class occupies one row in a sparse CSR matrix. Coordinates for absent or removed classes are zero. The identity entries represent self-similarity only in this feature matrix; they do not add graph edges or affect the quality calculations directly. Package names and AI cluster labels are not used as features. [D7]

The pilot on the middle release selects k=40 for n=554 classes, or approximately 13.85 classes per cluster. For each final release, k is scaled in proportion to its full graph's class count. This assumes that a similar average group size is useful across the history. It is a study design choice, rather than a lecture theorem or a guarantee of the best architecture, and it is not chosen by maximizing the final MQ scores. [D7; D8]

Selecting the seven dependency relationship types is a separate choice from selecting k. The course asks for the cluster-count choice to be explained, but does not prescribe a single-release elbow or this size-scaling rule. The one-pilot policy is a decision of this study. [AS p.4; L67 p.8; D8]

A3 and A4 use the same k for a release, except that k cannot exceed the number of retained classes. An empty graph uses k=0. The command-line option --k allows a separate exploratory run, while all 120 final partitions follow the rule above. The pilot also compares the algorithms at ACDC's cluster count to examine the effect of group size; that comparison does not select the final k. [D8]

The k-means settings are k-means++ initialization, Lloyd updates, random_state=484, n_init=10, max_iter=300 and tol=10^{-4}, with numerical thread pools limited to one. Class order and output labels use a fixed ordering. Each run records the actual cluster count, convergence warnings, iteration count, inertia and library versions. This also records cases where identical feature rows produce fewer than k clusters. [SK; D7]

3.3 Choosing the parameters in one pilot

The pilot uses **Guava 25.0**, the middle release in the selected sequence. Its position determined the choice before its quality scores were inspected. ACDC is tried with subgraph sizes 10, 20 and 40. K-means is tried on a fixed grid of k values on this release only. These pilot runs are kept separate from the 120 final A1–A4 architectures, whose parameters are not tuned independently for each release. [D8]

Pilot configuration	k	Largest	Singles	Cohesion	Coupling	MQ
ACDC -l10	110	49	23	0.324	0.003	0.320
ACDC -l20	110	49	23	0.324	0.003	0.320
ACDC -l40	110	49	23	0.324	0.003	0.320
KMeans (pilot elbow)	40	69	0	0.165	0.006	0.159
KMeans (matched k diagnostic)	110	28	4	0.268	0.005	0.264

The three ACDC settings produce the same required metrics on this release, so the final runs retain the middle setting, 20. This result describes the pilot; other graphs may respond differently to the parameter. [D8]

Giving both algorithms k=110 groups produces an MQ of 0.3202 for ACDC and 0.2636 for k-means. With the elbow-selected k, k-means has MQ 0.1589. Comparing these values helps show how much cluster count matters on this graph. Equal counts do not ensure equal size distributions or equally meaningful groups, so this diagnostic is not used to set the final k. [D8]

3.4 Reading the k-means elbow curve

Inertia is the sum of squared distances from classes to their cluster centers. Adding clusters generally reduces it; the elbow is a point beyond which further reductions become smaller. The pilot uses a specified geometric rule to locate that point, as shown below. [SK; D8]

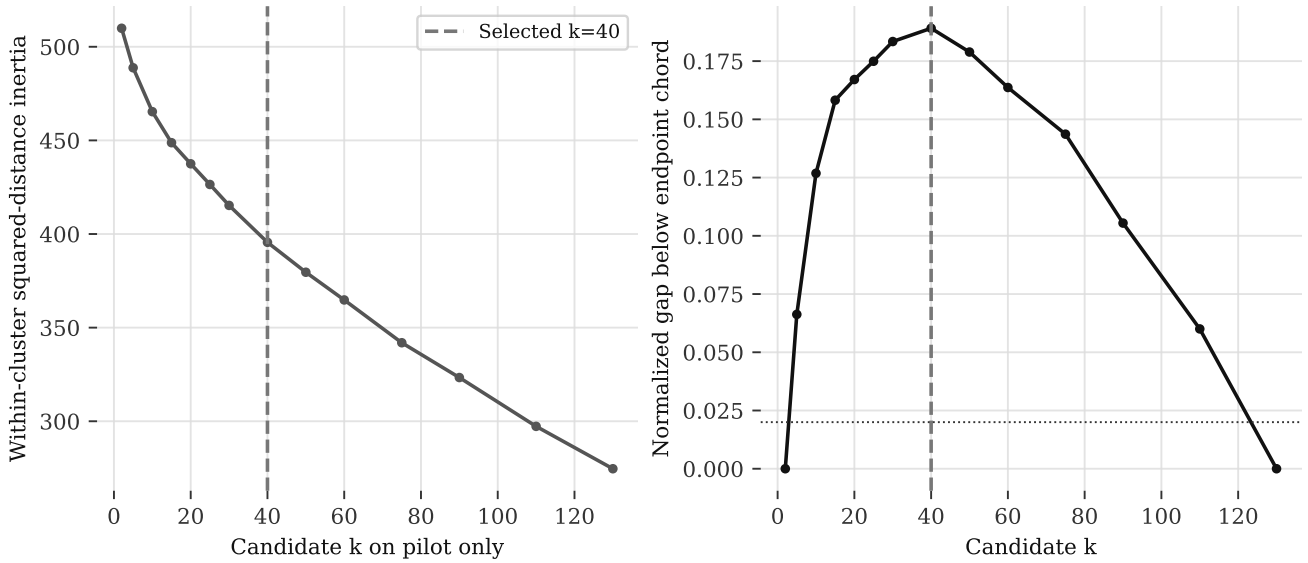


Figure 1. Pilot inertia and the normalized distance from the endpoint line. The k selection uses this curve, without using MQ. Source: results/pilot/k_inertia_curve.csv.

Candidate k	Inertia	Normalized gap	Decision
2	509.8831	0.0000	
5	488.7809	0.0663	
10	465.3418	0.1268	
15	448.7721	0.1582	
20	437.4963	0.1671	
25	426.4603	0.1749	
30	415.2801	0.1834	
40	395.5614	0.1891	Selected
50	379.5833	0.1789	
60	364.7866	0.1637	
75	341.9334	0.1436	
90	323.3453	0.1055	
110	297.2783	0.0600	
130	274.6421	0.0000	

The rule normalizes x=(k−k_min)/(k_max−k_min) and y=(I−I_last)/(I_first−I_last), then calculates gap=1−x−y. It chooses the largest interior gap if that gap is at least 0.02, resolving exact ties with the smaller k. Otherwise it uses the candidate nearest to the declared fallback, k=30. The recorded decision is maximum_normalized_chord_gap: k=40, with maximum gap 0.1891. [D8]

The selected elbow is broad: neighboring candidates have similar gaps. Its position also depends on the candidate range, the feature representation and the fitted clusters. It indicates diminishing improvement in distance fit on this graph, rather than a known number of true components. Scaling the chosen k by class count then carries the same target average group size to other releases without repeating the sweep. The results include the actual cluster sizes so that this assumption can be assessed. [D8]

3.5 Evaluating the recovered groups

A useful structural grouping should bring related classes together while keeping dependencies between groups relatively sparse. The reconstruction lecture expresses this through modularization quality (MQ): average cohesion within clusters minus average coupling between pairs of clusters. In this study, edges are directed and unweighted, with no self-loops. Let n_i be the number of classes in cluster i, μ_i its internal edge count, and ε_{ij} the edges between clusters i and j, counted in both directions. [L67 pp.15–17]

$$\text{cohesion}_i = \mu_i / [n_i(n_i - 1)] \text{ for } n_i > 1; \text{ otherwise } 0$$
$$\text{coupling}_{ij} = \varepsilon_{ij} / (2n_i n_j)$$
$$MQ = (1/k)\ \Sigma_i \text{cohesion}_i - (1 / \text{choose}(k,2))\ \Sigma_{i<j} \text{coupling}_{ij}$$

The denominator n(n−1) counts the possible directed edges within a cluster when self-loops are excluded. The lecture permits alternative density denominators; this report uses the one in its worked directed-graph example. A singleton has cohesion zero. A partition with one cluster has coupling zero, and an empty partition reports zero by convention. Pairs of clusters with no connecting edges still count in the coupling average. Stating these choices is necessary to reproduce the same MQ value from the same grouping. [L67 p.17; D9]

Checking the formula on a small graph

Take clusters C1={a,b} and C2={c,d}, with edges a→b, b→a, c→d, d→c and b→c. Each cluster has cohesion 2/(2×1)=1. Their coupling is 1/(2×2×2)=1/8, giving **MQ=1−1/8=0.875**. If an isolated singleton is added as a third cluster, average cohesion becomes 2/3 and average coupling becomes (1/8)/3=1/24. MQ is then 5/8. None of the original five edges has changed; the lower score follows from averaging over the additional cluster and cluster pairs. [D9]

Interpreting a score

For this graph model, MQ lies between −1 and 1. It describes how dependencies are concentrated within groups, but it cannot establish functional correctness, runtime performance, developer productivity or user-perceived quality. The averages also give a small cluster the same weight as a large one. Cluster count, cluster sizes and the treatment of isolates can therefore affect the result. The report includes cohesion weighted by class count, edge fractions and cluster-size distributions to help interpret the main score. [L67 p.16; L12 p.45; D9]

A1 and A3 are evaluated on the full graph, while A2 and A4 use the graph left after UE removal. Comparing a full-graph score directly with a filtered score changes both the groups and the classes being assessed. A further comparison addresses this by removing the same classes from the full partition and scoring both partitions on exactly the retained nodes and edges. MoJoFM compares the memberships of partitions; neither partition is assumed to be ground truth. [D3; D9; D16]

4. How Guava changes over time

4.1 Classes and dependencies over time

The first selected release has 317 classes and 1,289 dependencies. By the last release, these counts reach 594 and 2,750. The next figure shows the changes across all 30 samples. [D1]

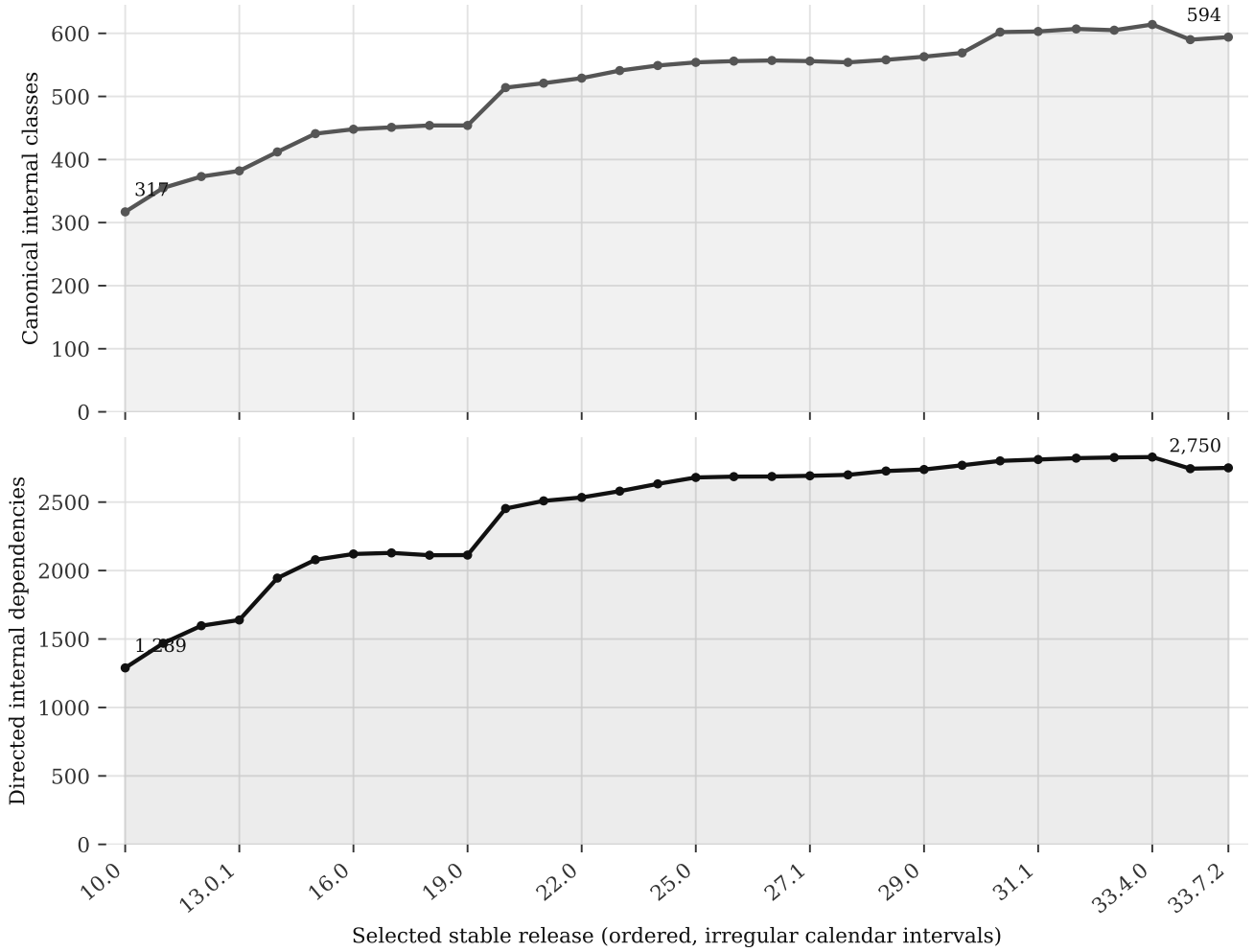


Figure 2. Project class counts and distinct directed dependencies for 30 major/minor release families. Dependencies come from the instructor's extractor. Source: results/sizes.csv.

Classes include project-owned annotation types and isolates, meaning classes with no recorded edges. Dependencies exclude annotation applications, following the instructor's extractor. This distinction matters when releases change compatibility or nullness annotations without comparable changes to executable dependencies. [D1; D6]

The samples are equally spaced along the axis, but the time between releases varies. The selection reduces repeated patch observations rather than sampling at regular calendar intervals. New functionality, implementation changes, compiler output and static-analysis coverage can all affect the counts. Growth alone cannot distinguish these explanations or show that design quality has worsened. [L12 pp.43–48; D4]

Largest changes in dependency count

Between 19.0 and 20.0, dependencies change from 2,113 to 2,453. The change-point files list the added and removed classes and edges so that this interval can be checked against the source. [D15]

Between 13.0.1 and 14.0, dependencies change from 1,639 to 1,945. The change-point files list the added and removed classes and edges so that this interval can be checked against the source. [D15]

4.2 Annotation types and isolated classes

A class can belong to Guava even when the extractor records no relation to or from it. The inventory retains these isolated nodes, including annotation declarations. The next figure and the endpoint table show how much of the inventory they represent. [D6]

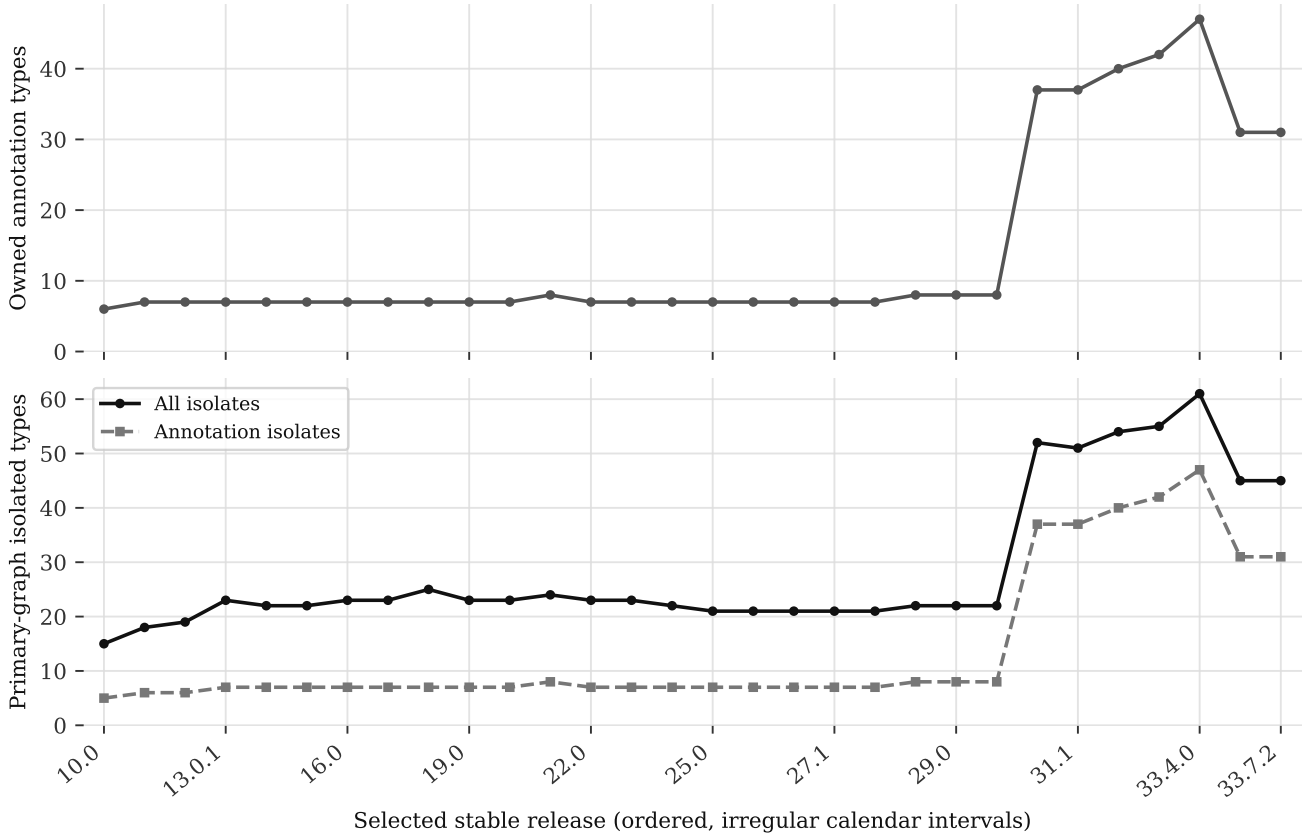


Figure 3. Project-owned annotation types and isolated nodes. Annotation declarations remain in the inventory; annotation applications are not edges. Source: classes.csv and dependencies.csv for each release.

Inventory category	10.0	33.7.2
All merged project-owned types	317	594
Nonannotation types	311	563
Annotation types	6	31
All isolated nodes	15	45
Isolated annotation types	5	31

Isolates affect clustering even though they contribute no edges. The ACDC adapter puts each isolate in a single-class cluster, whose cohesion is zero under the chosen metric. For k-means, an isolate still receives the identity component of its feature row. Annotation types can therefore influence class counts, cluster counts and MQ without introducing annotation-application edges. [D6; D9]

The categories overlap: an isolated annotation type is included in both the annotation and isolate totals. The rows should not all be added together. Lack of member-use evidence also does not justify removal by Usage Equality; the detector retains that case. [D2]

4.3 Changes between sampled releases

Total counts can hide replacement: one release may add classes while removing others. The next figure separates additions and removals and compares the dependencies and partitions of classes present in both adjacent samples. [D10; D16]

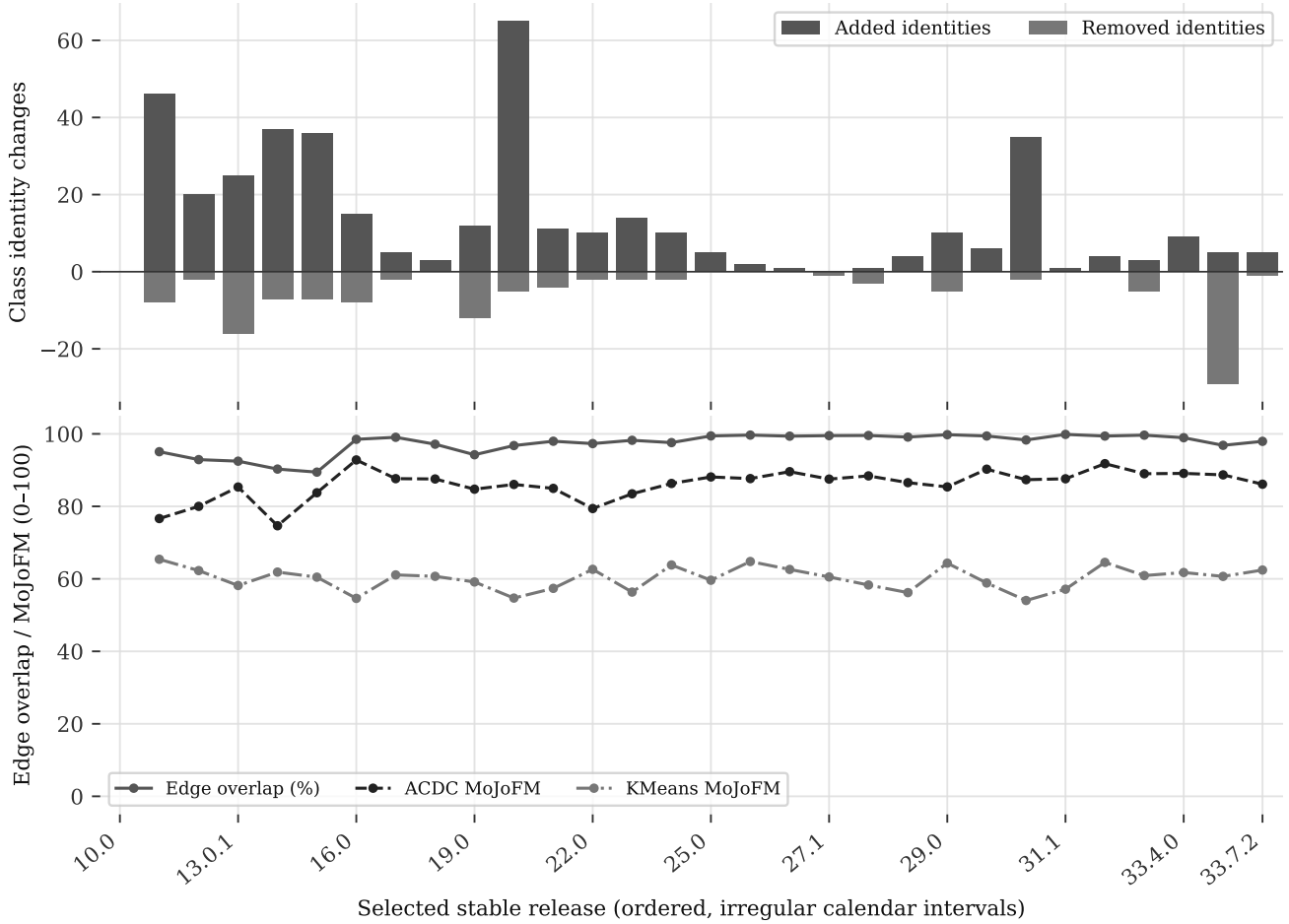


Figure 4. Class-name changes and structural agreement between adjacent samples. Edge overlap is Jaccard similarity multiplied by 100. MoJoFM compares the previous partition with the current one on shared classes. The intervals vary in length.

The largest turnover is between 19.0 and 20.0, with 65 class names added and 5 removed. Matching by name counts renames and package moves as well as additions and deletions. Compiler output or changes in what the model observes can also affect the inventory. These counts therefore do not directly measure features gained or lost. [D10]

The public-suffix implementation illustrates this distinction. It remains within the project boundary when it moves from com.google.common.net to com.google.thirdparty.publicsuffix, although a renamed class can still count as one removal and one addition. [D4; D6]

Edge Jaccard compares dependencies among shared classes. It can remain high while new or removed classes cause large changes in total edges. Temporal MoJoFM compares A1/A3 partitions restricted to those same shared classes. These measures describe stability between samples, rather than correctness against an intended architecture. [D10; D16]

4.4 Relating graph changes to the source

The four intervals below have the largest absolute percentage changes in dependency count. They were chosen for closer inspection after the release sample and measurements were fixed. The audit records every changed class and edge, together with source-JAR paths and hashes, so the observations can be traced to the published files. [D15]

Sample transition	Classes	Edges	Edge change
10.0 → 11.0	317 → 355	1289 → 1469	13.96%
11.0 → 12.0	355 → 373	1469 → 1597	8.71%
13.0.1 → 14.0	382 → 412	1639 → 1945	18.67%
19.0 → 20.0	454 → 514	2113 → 2453	16.09%

10.0 to 11.0

The inventory gains 46 class names and loses 8. The largest additions are in hash (17), collect (7), math (6). The largest removals are in collect (4), cache (2), util.concurrent (2). The graph gains 271 edges and loses 91. Among the classes present at both endpoints, edge Jaccard similarity is 0.9508. [D15]

Added examples: base.AbstractIterator, cache.AbstractLoadingCache, cache.ForwardingLoadingCache. Their exact source entries are listed in the change-point audit. [D15]

11.0 to 12.0

The inventory gains 20 class names and loses 2. The largest additions are in collect (8), reflect (7), base (2). The largest removals are in collect (1), hash (1). The graph gains 184 edges and loses 56. Among the classes present at both endpoints, edge Jaccard similarity is 0.9290. [D15]

Added examples: base.Absent, base.Present, cache.CacheBuilderSpec. Their exact source entries are listed in the change-point audit. [D15]

13.0.1 to 14.0

The inventory gains 37 class names and loses 7. The largest additions are in collect (19), io (8), reflect (4). The largest removals are in collect (5), base (2). The graph gains 383 edges and loses 77. Among the classes present at both endpoints, edge Jaccard similarity is 0.9029. [D15]

Added examples: cache.LongAddable, cache.LongAddables, collect.AbstractMapBasedMultimap. Their exact source entries are listed in the change-point audit. [D15]

19.0 to 20.0

The inventory gains 65 class names and loses 5. The largest additions are in graph (41), util.concurrent (7), math (6). The largest removals are in collect (2), io (2), util.concurrent (1). The graph gains 385 edges and loses 45. Among the classes present at both endpoints, edge Jaccard similarity is 0.9677. [D15]

Added examples: base.CommonMatcher, base.CommonPattern, base.ExtraObjectsMethodsForWeb. Their exact source entries are listed in the change-point audit. [D15]

Renames can appear as both a removal and an addition; added classes may support existing features. Annotation declarations can alter class and isolate counts without adding application edges. These counts alone do not show architectural improvement. [D6; D15]

5. Comparing the recovered architectures

5.1 Classes removed by Usage Equality

Removing one broadly used class can remove many edges. The next figure compares the number of selected classes with the share of dependencies connected to them. The thresholds are fixed across releases, and all selected classes are removed in one pass. [D1; D2]

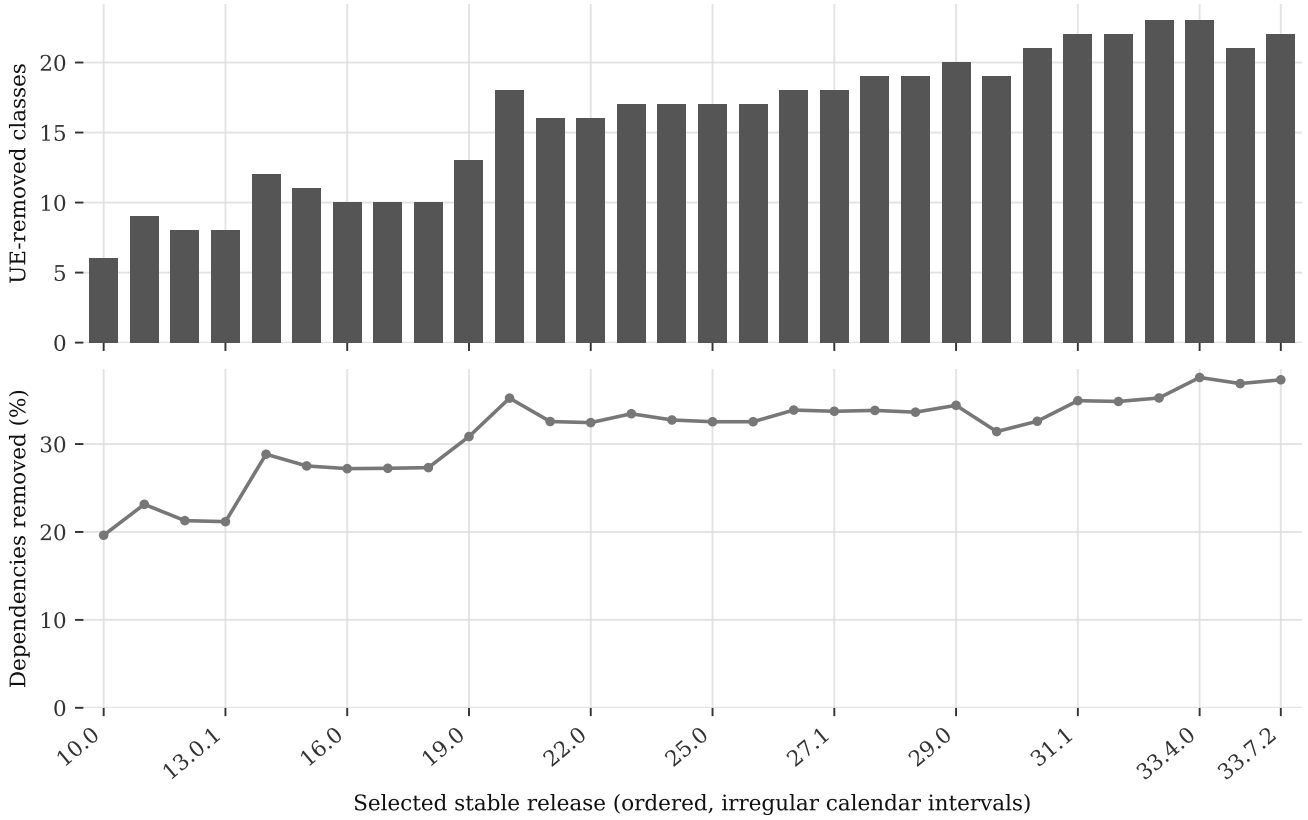


Figure 5. Classes selected by Usage Equality and the fraction of full-graph dependencies removed with them. Source: results/sizes.csv.

Most frequently removed classes	Releases selected / 30
base.Function	30
base.Preconditions	30
collect.Serialization	30
collect.UnmodifiableIterator	30

In the latest release, Usage Equality removes 22 of 594 classes, leaving 1,724 dependencies, or 62.7% of the full graph. Selection does not imply poor design: a shared utility can serve its purpose well while making subsystem boundaries harder for a clustering algorithm to distinguish. [UE; D1; D2]

The frequency table uses all 30 releases as its denominator, including releases before a class first appears. The per-release ue.json and omnipresent.csv files distinguish absent classes, retained candidates and selected classes. [D2]

Two class-level checks

The following cases show why many callers are not enough to trigger removal. The columns e and c are the Usage Equality and cardinality scores defined in the method section; both must meet their thresholds. [D14]

Latest audited class	Callers	Members	e	c	Decision
base.Preconditions	312	29	0.9276	0.9968	Remove
collect.Iterators	50	41	0.4980	0.9800	Retain

Preconditions meets both thresholds. Iterators has many callers, but Iterables dedicates a comparatively large share of its member references to it. The resulting maximum dedication of about 0.502 prevents removal. The independent audit records the exact fractions, every caller's scores and the supporting member-CSV rows. [D14]

5.2 Modularization quality

The next figure shows MQ for the four conditions, with each series summarized below it. A1 and A3 use the full graph; A2 and A4 use the filtered graph. The algorithms can therefore be compared on matching graphs. Comparing before and after filtering requires the additional check described later. [D1; D3]

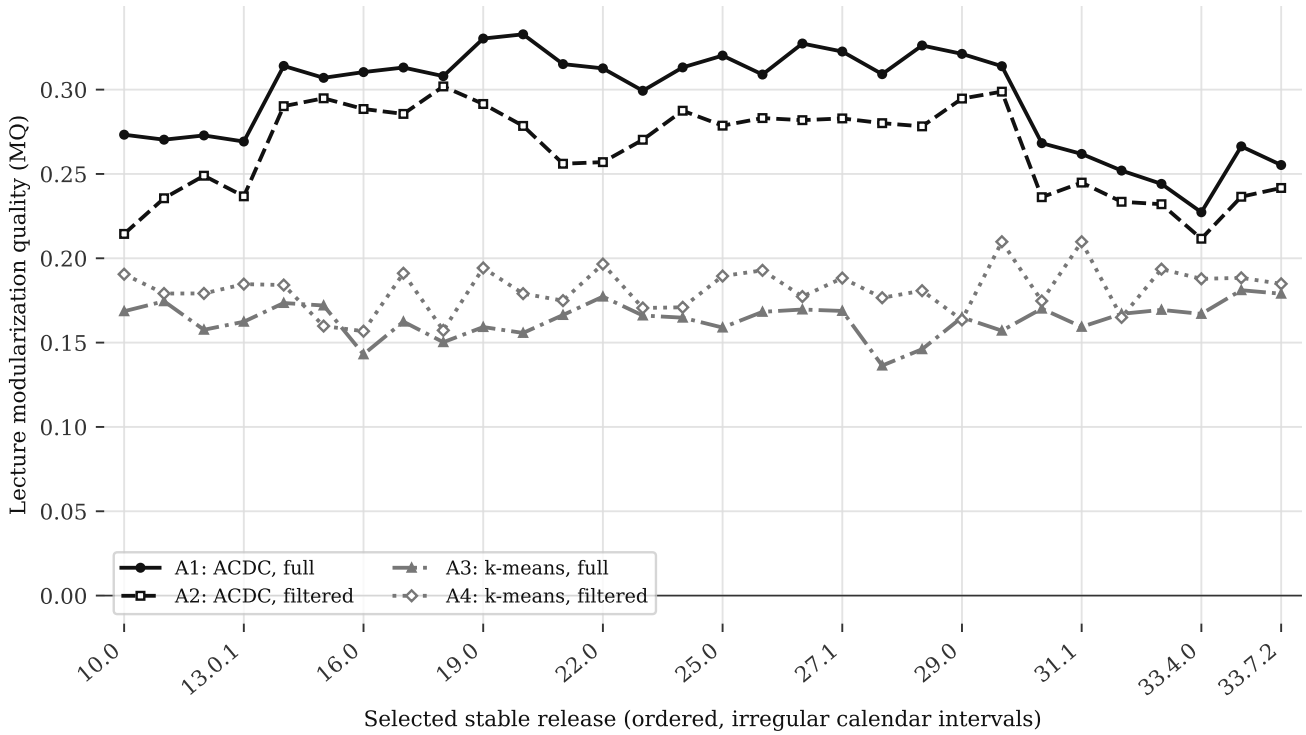


Figure 6. MQ for the four conditions. A1/A3 use full graphs and A2/A4 use filtered graphs. A gap between full and filtered scores includes a change in the evaluated graph.

Condition	Minimum	Median	Maximum	Latest
A1: ACDC, full	0.227	0.309	0.333	0.255
A2: ACDC, filtered	0.212	0.278	0.302	0.242
A3: k-means, full	0.136	0.166	0.181	0.179
A4: k-means, filtered	0.157	0.182	0.210	0.185

ACDC scores higher than k-means in 30 of the 30 full-graph comparisons and 30 of the filtered-graph comparisons. The difference depends on the metric and on how finely each algorithm divides the classes. It does not establish agreement with a developer-approved architecture. Cluster sizes are examined below to help interpret the scores. [D1; D9]

5.3 Cohesion and coupling

MQ subtracts average coupling from average cohesion. Showing both terms helps explain whether a change comes from denser internal groups or different relationships between groups. The next figure uses a separate vertical scale for each term. [D9]

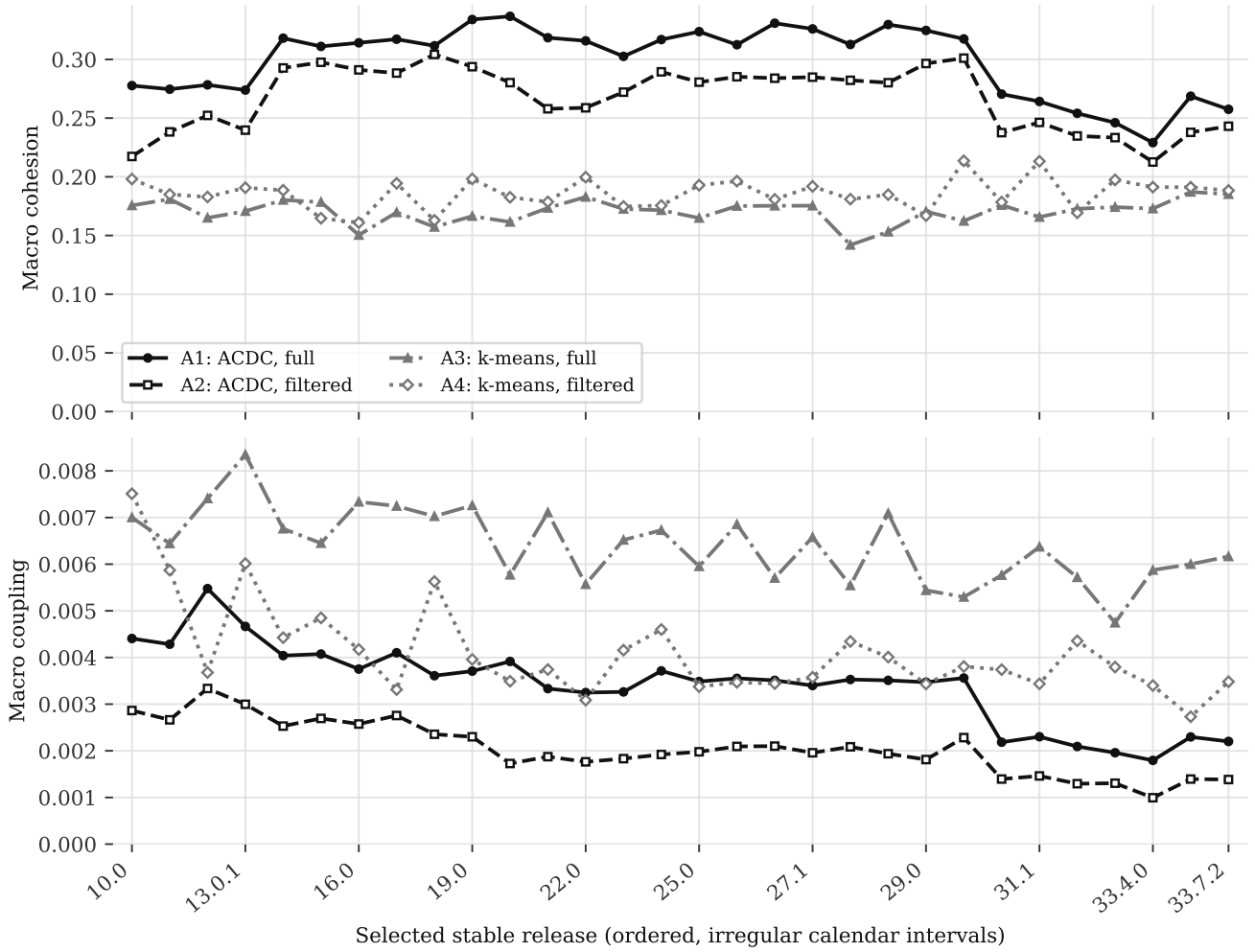


Figure 7. Average cohesion and coupling. Cohesion averages over clusters; coupling averages over unordered cluster pairs. The vertical scales differ.

These are averages over clusters and cluster pairs, not individual classes or edges. Cluster sizes and the number of pairs therefore affect the result. In particular, low average coupling need not mean that few dependencies cross cluster boundaries. [D9]

An appendix result can be reconstructed from its A<n>_quality/per_cluster.csv and per_pair.csv files: sum the normalized terms, divide by the respective numbers of clusters and unordered pairs, then subtract coupling from cohesion. Values in the report are rounded; the CSV and JSON files retain the calculation precision. [D9]

5.4 The effect of filtering on the grouping

To separate regrouping from a change in the measured graph, the original partition is first restricted to the retained classes. This removes selected classes without regrouping the survivors; empty clusters disappear. That partition and the newly clustered filtered partition are then scored on exactly the same retained edges. [D3]

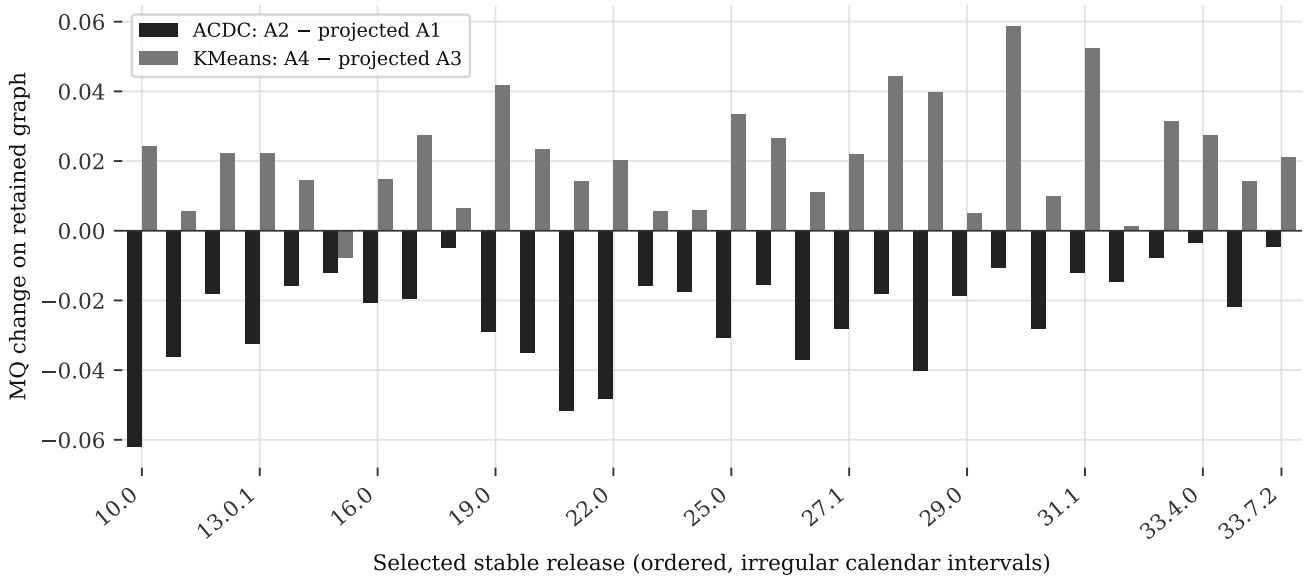


Figure 8. MQ difference between clustering the filtered graph again and keeping the original grouping of retained classes. Positive values favour reclustering. Source: results/same_graph_comparisons.csv.

Algorithm	Median ΔMQ	Latest ΔMQ	Δ > 0	Δ < 0
ACDC	-0.0191	-0.0047	0	30
k-means	0.0215	0.0211	29	1

Here, ΔMQ is the score after reclustering minus the score for the original grouping of retained classes. Holding the graph fixed makes this a comparison of the groupings produced by the stated recovery procedure. It does not establish that filtering helps every algorithm or software system. [D3]

5.5 Agreement between partitions: MoJoFM

MQ describes a partition's structure; MoJoFM measures how similar two partitions are. Before comparing a full-graph result with a filtered result, both are restricted to the same retained classes. The next figure shows the lecture's agreement measure over the sampled history. [L67; D16]

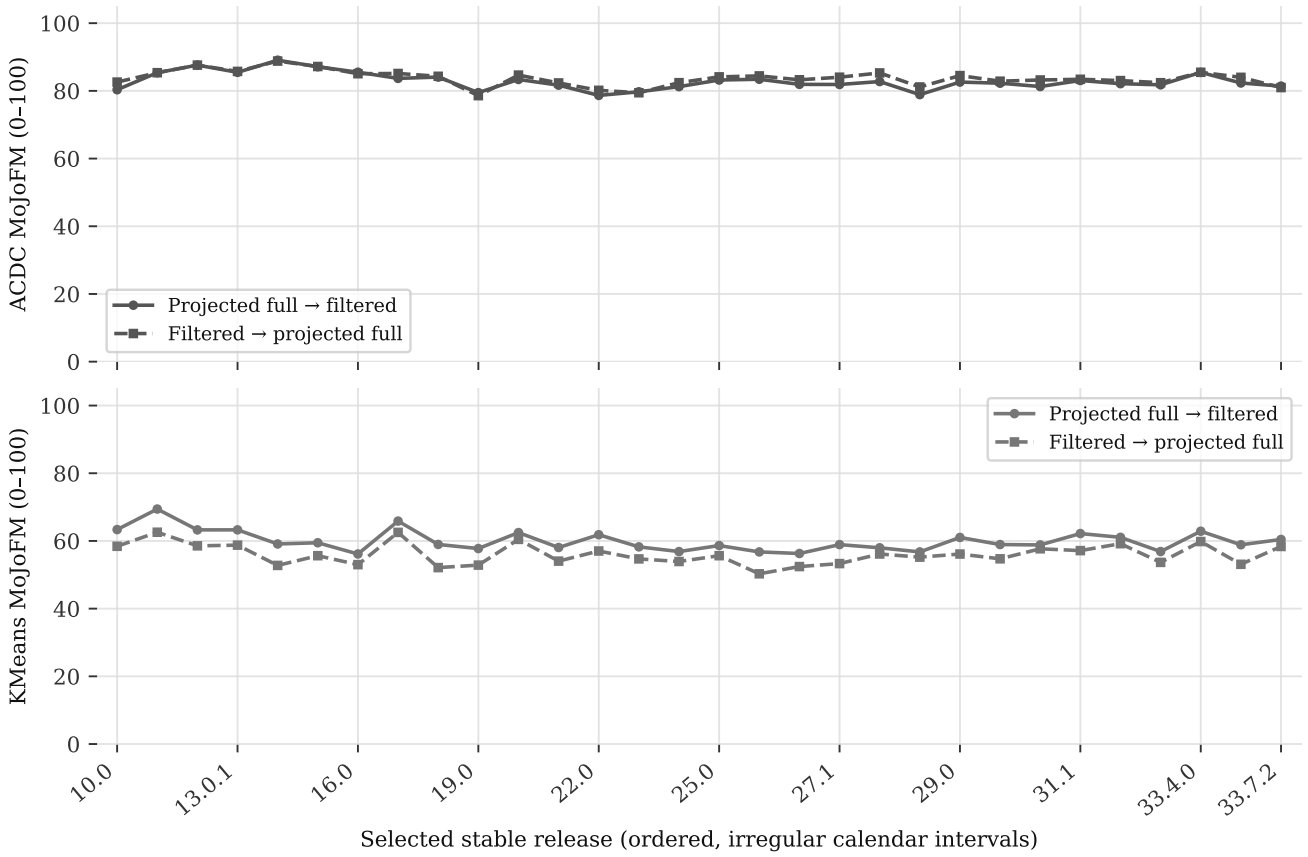


Figure 9. MoJoFM agreement on the retained class set, on a 0–100 scale. Forward and reverse comparisons use different reference partitions. [D16]

Latest comparison	Nodes	Forward MoJoFM	Reverse MoJoFM
A1 → A2	572	81.41	80.98
A3 → A4	572	60.44	58.36
A1 → A3	594	43.71	41.11
A2 → A4	572	47.19	42.78

$$\mathrm{MoJoFM}(P \to Q) = 100 \times [1 - \mathrm{MoJo}(P,Q) / \mathrm{maxMoJo}(Q)]$$

MoJo counts the fewest object moves and cluster joins needed to transform partition P into reference partition Q. The normalizer depends on Q, so reversing the comparison can change MoJoFM. Saved results include both directions, distances, denominators, cluster counts and a hash of the exact class set. Normalized scores for empty or one-class sets are recorded as undefined. [L67; D16]

An expert reference is needed to interpret this score as recovery accuracy, but not to measure agreement between two stated partitions. MoJoFM does not assess module names or responsibilities. Temporal and AI comparisons use the same common-class rule. [D16]

5.6 Cluster sizes and the interpretation of MQ

MQ differences can partly reflect one algorithm producing many small clusters and another producing fewer large ones. The next figure shows the number of clusters and the fraction of classes in the largest one. The table adds details for the latest release. [D9]

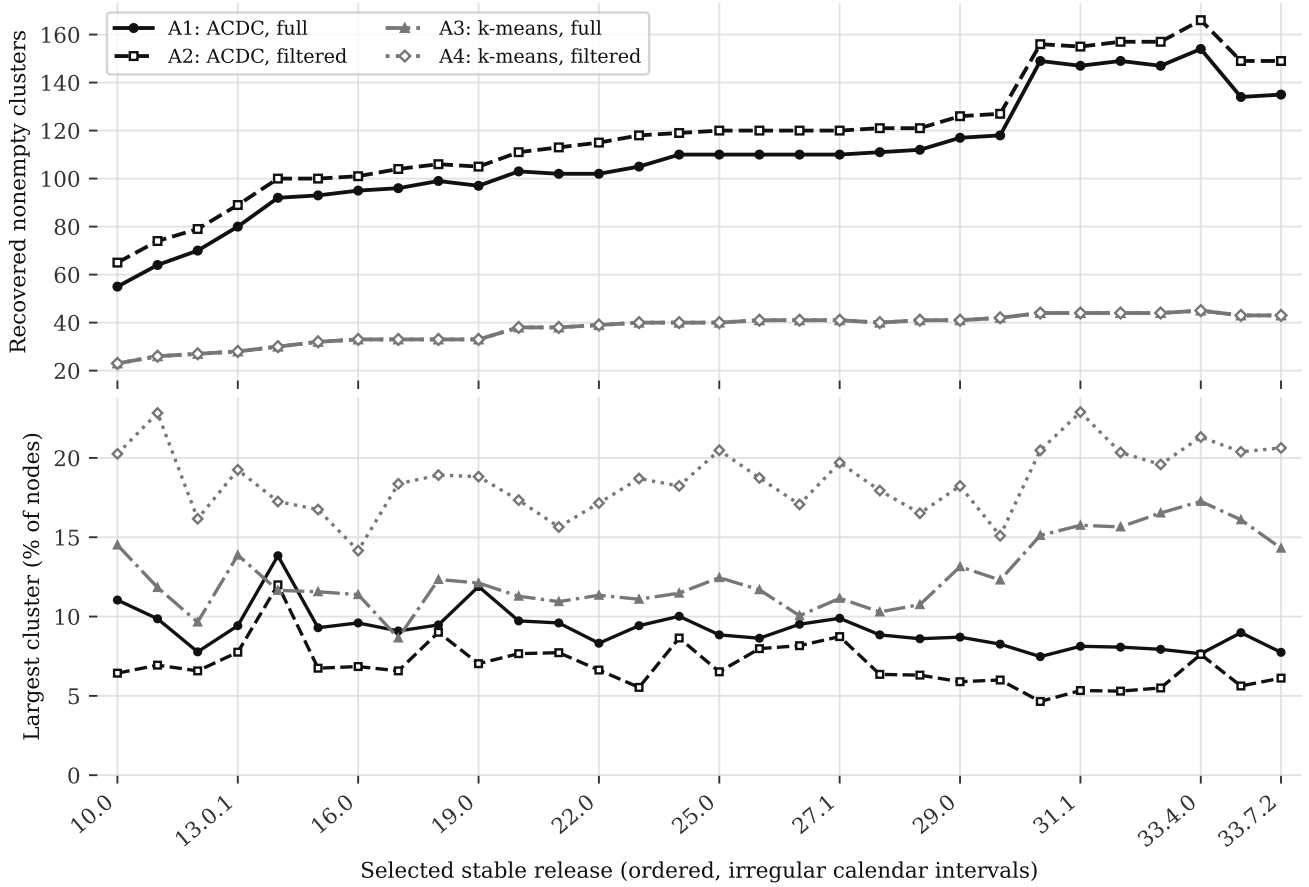


Figure 10. Number of clusters and fraction of classes in the largest cluster. ACDC's subgraph-size parameter does not cap the final cluster size.

Latest	k	Largest	Singletons	Size-weighted cohesion	Inter-edge share
A1	135	46	47	0.232	63.9%
A2	149	35	60	0.231	53.1%
A3	43	85	0	0.129	65.9%
A4	43	118	1	0.136	51.5%

Size-weighted cohesion gives larger clusters more influence in proportion to their class counts. The lecture's MQ instead weights each nonempty cluster equally. Inter-edge share is the fraction of all dependencies crossing cluster boundaries. These additional measures help reveal large residual groups or numerous small clusters that averages can obscure; they do not replace the required MQ calculation. [D9]

5.7 Interpreting the largest recovered groups

A cluster's members help explain what the algorithm has grouped together. The tables below show the five largest clusters from ACDC and k-means on the latest full graph, along with their most common packages and example classes. Package names are added after clustering to aid interpretation; they were not algorithm features. [D7]

A1: ACDC, full

Cluster	Classes	Main packages; example classes
acdc_0011	46	hash: 24; primitives: 7; io: 5 base.Defaults, base.Preconditions, base.Utf8
acdc_0023	46	collect: 41; base: 2; math: 1 base.Predicate, base.Predicates, collect.AbstractListMultimap
acdc_0015	21	util.concurrent: 20; base: 1 base.Function, util.concurrent.AbstractCatchingFuture, util.concurrent.AbstractTransformFuture
acdc_0064	19	graph: 16; collect: 3 collect.HashBiMap, collect.HashMultiset, collect.UnmodifiableIterator
acdc_0123	18	util.concurrent: 18 util.concurrent.AbstractListeningExecutorService, util.concurrent.Callables, util.concurrent.DirectExecutor

A3: k-means, full

Cluster	Classes	Main packages; example classes
kmeans_0000	85	collect: 19; base: 12; util.concurrent: 12 annotations.Beta, annotations.GwtCompatible, annotations.GwtIncompatible
kmeans_0005	46	io: 9; collect: 8; primitives: 8 base.Defaults, base.PairwiseEquivalence, base.Utf8
kmeans_0014	26	collect: 26 collect.AbstractListMultimap, collect.AbstractMapBasedMultimap, collect.AbstractMultimap
kmeans_0040	23	util.concurrent: 23 util.concurrent.AbstractCatchingFuture, util.concurrent.AbstractExecutionThreadService, util.concurrent.AbstractFuture
kmeans_0023	22	collect: 21; util.concurrent: 1 collect.CollectCollectors, collect.DescendingImmutableSortedMultiset, collect.FluentIterable

Cluster identifiers are output labels, not established subsystem names. ACDC's saved original_containers map links these labels to the implementation's container names, including any orphan container. This matters when interpreting a large group that may contain classes left over from the main recovery steps. Complete assignments are available for every condition and release. [D7]

6. The AI-assisted architecture proposal

6.1 An AI interpretation of Guava 33.7.2

The AI proposal below is the saved model response for the latest release. Its input checksums match the class and dependency files used in this study. The recorded assignment is evaluated against the current A1–A4 results, and its exact prompts and response provide the experiment record. Regenerating this report recalculates no architecture and makes no new request to the model. [D12]

The final experiment asks an AI model to propose a decomposition of Guava 33.7.2, the latest release in the study. The model was ChatGPT GPT-6 Astra Pro (fresh delegated session; no seed control exposed). It first outlined a possible architecture using class names and package counts. It then examined the dependency graph and package documentation before assigning every class to a module. This produced one final architecture; the earlier outline was retained to show how its reasoning changed. [AS pp.3–4; D12]

What the model received

The graph supplied to the model contains 594 classes and 2,750 directed dependencies. The package documentation comes from the source JAR for the same release. The files below record these inputs; the checksum prefixes identify the exact copies used. [D12]

Input file	Bytes	SHA-256 prefix
classes.csv	80,871	4351db3d545cd194
dependencies.csv	224,223	2ec951b2d5915223
package_counts.json	629	bfa90ac28edd6ca8
package_dependencies.csv	3,465	6a4670af58a29fde
package_documentation.txt	24,048	612c261362cb659e

The model was given no ACDC or k-means partitions, usage-equality decisions or metric results. It could not browse external sources. Its final assignment had to place each class in exactly one module before the evaluation script calculated quality. The saved experiment record contains the full checksums, file paths, activity log and outputs. [D12]

How to interpret this comparison

The experiment record sets out three limits on the comparison and its reproducibility:

Names and package documentation are allowed AI evidence; structural algorithms receive graph features only. This information asymmetry prevents a controlled same-input superiority claim.

A fresh session still has general pretrained knowledge. Hidden runtime instructions and model nondeterminism cannot be fully reproduced.

Original revision AI output is preserved in the original Git commit, not reused as this response.

The initial outline is saved in initial_hypothesis.md and the final explanation in response.md. The sections that follow give the prompt, the proposed modules and their evaluation. The scores can be recalculated from the saved assignment. Asking the model again may produce a different architecture. [D12]

Additional instructions given to the model

The additional recorded instruction appears below, unchanged. It contained no baseline partition or quality score. No completion or status follow-up was saved for this experiment. The main task prompt follows in the next section. [D12]

Launch instruction: Working directory /workspace/scratch/7500a23b5952/software-evolution-ex1. This is a fresh controlled AI architecture-recovery experiment. Your exact task is the full contents of results/ai/prompt.txt. Read that file first and follow it strictly. Read only that prompt and the listed results/ai/inputs evidence; do not inspect any other project data, results, code, baseline architectures, UE decisions or numeric quality scores. First save the initial names-only hypothesis, then inspect the remaining evidence and produce the specified final outputs. Your launch identity is GPT-6 Astra Pro as inherited. The launcher supplies no algorithm outcomes. Complete the recovery now; an independent reviewer evaluates afterward.

SHA-256: d5348c0760560ec08afdd8dcf8b974d39d2938aaa62ea24be97c2e908ae120e5

6.2 The experiment prompt

This is the saved prompt, reproduced verbatim so that the task given to the model can be checked. The record does not include hidden platform instructions or model internals. [D12]

Recover a maintainable conceptual module architecture for the latest selected Guava JRE release, 33.7.2, from the supplied evidence. This is an actual recorded AI architecture-recovery experiment for EPL484 Software Evolution. You are a fresh AI reviewer and must not inspect any ACDC/KMeans partitions, UE decisions, metric outcomes, reports, or other experiment outputs.

Use only the following files under results/ai/inputs (all from the same pinned release): classes.csv (594 canonical outer types), dependencies.csv (2750 binary directed internal class dependencies), package_counts.json, package_dependencies.csv, and package_documentation.txt (unmodified package documentation from the matching source JAR). The primary dependencies are the instructor DependencyExtractor relationships. External libraries have already been excluded and inner/anonymous classes merged. The inventory includes all owned types, including isolated classes and annotation declarations; annotation applications are not edges in this graph. Both com.google.common and Guava-owned com.google.thirdparty.publicsuffix are in scope. A dependency points from a class using a type/member to the class being used. Package declarations and documentation are evidence, not a validated intended architecture. Do not browse external sources or inspect other project files. You may use Python to read and summarize only these inputs and produce the requested outputs.

First inspect classes.csv and package_counts.json only. Record a concise initial architecture hypothesis in results/ai/initial_hypothesis.md before reading dependency/documentation inputs. This hypothesis is qualitative, not a second finalized architecture. Then read the remaining evidence and refine the hypothesis into ONE complete final class partition. Select a practical number of named responsibility modules; do not target a numeric clustering metric. Classes in one package may be split if responsibility boundaries and dependency evidence justify it. Preserve isolated types. Give every supplied class exactly one module and invent no classes. Do not place a class in multiple modules, including utilities.

Write these outputs under results/ai only:
1. assignment.csv with columns class,cluster, containing every supplied class exactly once. Use semantic stable cluster IDs.
2. architecture.json with schema_version, subject.release, modules (id, name, responsibility, exact class_count, boundary_rationale), cross_module_observations, changes_from_initial_hypothesis, and limitations. Cite concrete class/package relationships and graph counts where relevant, separating interpretation from observation.
3. response.md: complete final explanation, including how additional evidence changed or confirmed the names-only hypothesis; package-structure bias; confidence limits; and which intended-design claims need maintainer validation. Do not claim superiority or architectural accuracy without ground truth. Explain how the proposal helps a human understand Guava.
4. activity.json: exact input paths read, tool/method summary, model identity available from your runtime (do not invent unavailable identifiers), and confirmation that baseline partitions and numeric quality scores were not consulted.

Validate partition coverage against classes.csv. Do not compute MQ/cohesion/coupling or compare with baseline algorithms; the coordinator evaluates afterward on matching graph scopes. Leave unrelated files untouched. No subdelegation is needed for this small controlled experiment.

Prompt SHA-256: f9cb3f65ff158253eb15acb2580e8f2b4adbd7fd337307b1a84c6c79c218810a

6.3 The proposed modules

The model grouped all 594 classes into 18 modules, with each class assigned exactly once. The table summarises the responsibilities it gave those modules. These descriptions express the model's reading of the names, documentation and dependencies; they have not been endorsed by Guava's maintainers. The full architecture file explains each proposed boundary and the observations behind it. [D12]

Module	Classes	Responsibility excerpt
Core values and programming contracts	57	Basic values, argument checks, functional adapters, text helpers, timing, reference lifecycle support, and shared API/platform annotation declarations.
Core collections and ordering	127	Ordinary maps, bimaps, lists, sets and queues; immutable implementations of those types; collection views, iteration, ordering, forwarding decorators,…
Multimaps and multisets	59	Multiple-value mappings and occurrence-counting collections, including their interfaces, mutable and immutable implementations, filtering views, sorted…
Two-dimensional tables	16	Row/column-keyed Table APIs, mutable and immutable representations, forwarding views, and table-specific collectors.
Ranges and interval collections	15	Range endpoints and bounds, discrete domains, contiguous sets, and range-set/range-map representations.
Caching and removal policy	22	Cache APIs, loading, construction and configuration, eviction/removal policy, statistics, and local cache implementation support.
Futures and execution runtime	58	Future values and composition, listening executors, task submission and sequencing, executor adapters, thread factories, execution exceptions,…
Service lifecycle management	8	Service state/lifecycle APIs, service implementation templates, service groups, and lifecycle listener dispatch.
Synchronization and execution limits	15	Atomic state utilities, locks and monitors, striped/cycle-detecting synchronization, rate limiting, and time limits.
Graph models and traversal	53	Graph, network and value-graph APIs, builders, mutable/immutable representations, adjacency/connection machinery, and traversal.
Hash functions and probabilistic membership	33	Hash functions and codes, hashers, primitive sinks/funnels, digest and checksum adapters, hashing streams, and Bloom filters.
Streams, files and resource I/O	37	Byte/character sources and sinks, streams and readers, file/resource operations, resource closing, line processing, and base encoding.
Primitive value utilities	25	Primitive-type utilities and conversions, unsigned value types, immutable primitive arrays, and associated platform helpers.
Arithmetic and statistics	16	Arithmetic and rounding over numeric values, quantiles, statistical summaries/accumulators, and linear transformations.
Network identifiers and protocol values	10	IP addresses, host/port specifications, domain names and owned public-suffix support, media types, and HTTP constants.
Text escaping policies	16	Generic character/Unicode escape abstractions, escaper builders, and HTML, XML, and URL escaping policies.
Reflection and generic types	16	Generic type capture/resolution, type tokens, invocation and parameter inspection, type-to-instance maps, class-path inspection, and reflection helpers.
Event subscription and dispatch	11	Event bus APIs, synchronous/asynchronous dispatch support, subscription declarations, subscriber discovery, and exception handling.

Every class assignment is listed in results/ai/assignment.csv. The explanation and initial outline are in response.md and initial_hypothesis.md in the same directory. The module definitions, revisions and limitations are also recorded in architecture.json. [D12]

6.4 Comparing the AI proposal with ACDC and k-means

The AI proposal was evaluated with the same cohesion, coupling and MQ calculations as the other partitions. Its complete assignment uses the full graph, allowing comparison with A1 and A3. For comparison with A2 and A4, the removed classes were deleted from the same assignment and the scores were recalculated on the retained graph. The model was not asked to produce another architecture. [D12]

Partition	Graph	k	Cohesion	Coupling	MQ	Inter-edge %
AI final	Full	18	0.0966	0.0023	0.0943	40.7%
A1	Full	135	0.2575	0.0022	0.2553	63.9%
A3	Full	43	0.1852	0.0062	0.1790	65.9%
AI projected	Retained	18	0.0907	0.0012	0.0895	26.2%
A2	Retained	149	0.2431	0.0014	0.2417	53.1%
A4	Retained	43	0.1883	0.0035	0.1848	51.5%

Scores should be compared within the same graph: the full-graph rows form one comparison and the retained-graph rows form another. MoJoFM then shows how closely the class groupings agree. The two MoJoFM columns report each direction separately. [D12]

Baseline	Graph	AI → baseline MoJoFM	Baseline → AI MoJoFM
A1	full	28.57	61.63
A2	retained	32.85	63.18
A3	full	32.52	61.63
A4	retained	36.12	66.25

How the proposal changed after inspecting the evidence

The following three examples are taken from the model's saved explanation. They show how the graph and documentation affected its initial outline. [D12]

Four responsibilities: collections_core, multimaps_multisets, tables, and ranges. Documentation names multiple collection contracts and range semantics; direct references between Multimap and Multiset support their joint family, while table-specific implementations and collectors support a separate Table responsibility. Shared helpers and immutable representations prevent interpreting the split as clean compilation layers.

async_execution, service_lifecycle, and synchronization_limits; two blocking forwarding adapters join collections_core. Futures and executor composition share direct implementation relationships; lifecycle templates and ListenerCallQueue form a recognizable responsibility; atomic/lock/rate/time APIs express coordination. The two forwarding adapters directly reuse core collection forwarding types.

Separate primitive_values and mathematics. Documentation distinguishes representation/storage from arithmetic, and all five observed math/primitives cross-package edges point toward primitive utilities.

What the comparison shows

The AI proposal has MQ 0.0943 on the full graph and 0.0895 on the retained graph. It uses 18 modules, compared with 43 clusters for A3. AI → A3 MoJoFM is 32.52, and A3 → AI is 61.63. These measurements describe different aspects of the output: MQ summarises the pattern of dependencies within and between groups, while agreement measures compare the actual class assignments. [D9; D12]

The methods also work at different levels of detail and receive different information. The AI can use names and source documentation to propose semantic responsibilities; the structural algorithms follow rules defined over the dependency graph. The results therefore do not establish which method best recovers Guava's intended design. Nor do the proposed modules establish an acyclic decomposition suitable for separate builds. [D9; D12]

The difference between the two MoJoFM directions is substantial here: transforming the AI proposal towards A1 gives 28.57, while the reverse gives 61.63. Merging several fine groups may require joins, while separating a coarse group may require many individual moves. The target partition also determines the normalization. Both directions are needed to describe the comparison. [D12; D16]

The calculations are saved in results/ai/comparisons.csv, metrics.json and retained_metrics.json. The model's full account of dependencies between modules and its own limitations remains in architecture.json and response.md. [D12]

7. Discussion and practical limits

7.1 Discussion

What has been reconstructed?

The study reconstructs architecture by grouping classes from a static dependency graph. It therefore describes possible higher-level views of Guava's implementation. This distinction matters in the architecture-evolution lecture, which separates implemented, intended and described architecture, and static and dynamic views. To demonstrate architectural erosion, the study would also need an intended design rule and evidence that later changes violate it. A fall in MQ alone cannot establish that. [L5 PDF pp.9,12,14,18–19]

Relating the changes to Lehman's laws

The changes in class and dependency counts are consistent with continuing change, and the source examples help explain what changed between releases. They also provide a starting point for investigating structural complexity. Applying the wider claims in Lehman's laws requires further evidence: functional growth needs information about requirements and features; perceived quality needs user evidence; and claims about effort or organisational stability need process data. The lectures apply these laws to E-type systems. Applying them to Guava therefore also requires an explanation of how the library adapts to changing real-world needs. [L12 pp.33–48]

Why the definitions matter

The requirements lectures emphasise clear, measurable statements and the ability to trace a conclusion back to its evidence. That is why the report defines what counts as an internal class, a dependency, a stable release, an omnipresent class and a quality score. For example, a dependency in a compiled class file is a different observation from a relationship between requirements or an interaction observed while the program runs. Saving the inputs, parameters and per-cluster calculations makes these choices and their consequences checkable. It does not, by itself, make the choices correct. [L3 pp.9–19,27–33; L4 pp.11–15]

Limitations

The 30 releases were selected systematically from the eligible history. They are a sparse sample, so changes between the selected releases may be missed. The analysis also has a deliberate boundary: it covers the chosen Guava artifact and source namespaces, excluding external libraries and other modules. The findings describe that boundary rather than every interaction in an application that uses Guava. [D2; D4]

A static graph cannot capture every reflective call or dependency created at runtime. Differences in compilers and inlining can also affect published bytecode. The usage-equality detector depends on resolving member use and on its chosen thresholds; it implements a different method from the older Noise detector. The structure recovered by ACDC depends partly on how finely it divides the graph, while k-means depends on the feature geometry and the way the pilot's k rule is applied to other releases. [D4; D6; D7]

For the latest release, 659 member-use events could not be resolved (494 distinct recorded references); 335 of these events cross class boundaries after inner and anonymous classes have been merged. The diagnostics identify a gap in the member-use evidence. Dividing these counts by all recorded calls would mix external and intraclass events, so that ratio would not measure extraction completeness or the false-negative rate. [D14]

7.2 Running time and memory use

The usage-equality detector took a median of 0.0530 seconds per release, including CSV input. The figure compares the study timings with a separate benchmark on larger generated graphs. [D11]

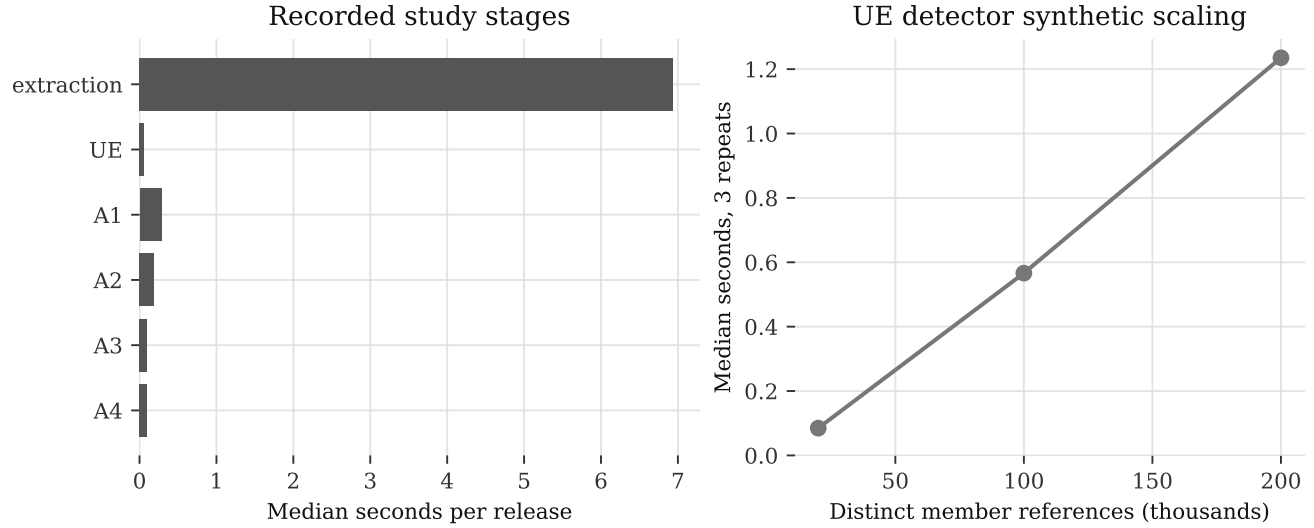


Figure 11. Time spent on the study stages and on the separate usage-equality benchmark. The two experiments measure different workloads.

Study stage	Median per release (s)	Maximum (s)	Total for 30 releases (s)
extraction	6.9338	7.8621	189.912
UE	0.0530	0.1053	1.666
A1	0.2852	0.3469	8.143
A2	0.1876	0.2124	5.523
A3	0.0995	0.1148	2.800
A4	0.0916	0.1499	2.639

Extraction includes Java startup and file I/O; clustering covers each adapter call. Downloads, the pilot, tests and report generation are excluded. The totals therefore represent the measured stages, rather than elapsed time for the entire project. [D11]

Testing larger inputs

Generated graphs have degree 20. Each timed detector call includes validation and diagnostics, and every node's expected score is checked. The table gives medians from three repeats. [D11]

Classes	Member references	Median time (s)	Peak process memory (MiB)
1,000	20,000	0.0847	34.0
5,000	100,000	0.5662	117.6
10,000	200,000	1.2353	221.8

Peak resident memory is cumulative for the process, including generated inputs and objects from earlier runs; it is not memory per detector call. Times depend on the machine and software. The results cover the tested sizes only: no speedup or correctness comparison with the old JNode executable was made. [D11]

8. Reproducing the analysis

8.1 Running the study again

The repository contains the inputs, code and saved results needed to repeat the analysis. First install the Python package versions and Java tools listed in the README. Then use the commands below to unpack the saved inputs, check them, rebuild the analysis and create the report. The --offline option checks the existing artifact cache; the acquisition guide also explains how to download the files again.

```
python scripts/unpack_data.py
python scripts/select_versions.py --check
python scripts/fetch_data.py --offline
python scripts/fetch_tools.py
python scripts/build_java.py
python scripts/run_study.py --phase all
python scripts/audit_change_points.py
python scripts/evaluate_ai.py
python scripts/validate_results.py
python scripts/build_report.py
```

Following a result back to its input

For a particular release, start with its row in results/metrics.csv. Its assignment file shows which cluster contains each class, and its graph files show the dependencies used to calculate the score. The version manifest then leads back to the downloaded release. The main files are listed below. [D1; D5; D9]

Artifact path	What it contains
config/versions.json	30 distinct families with immutable representatives, URLs, hashes and catalog decisions.
data/graphs/<version>/	Original instructor input/raw output; normalization; typed and binary edges; aliases; separate supplemental member evidence and comparisons.
results/versions/<version>/	UE scores, retained graph, exactly one A1–A4 assignment and its execution/quality details.
results/*.csv	30 size rows, 120 metric rows, 60 same-graph rows, 29 transitions, 180 stage timings, 178 directional comparison pairs.
results/run_manifest.json	Input/config hashes, algorithm-source hashes, Python, Java and package versions.
results/ai/	Actual AI prompts, provided input, output, coverage audit and comparison.
report/report_manifest.json	Hashes of files read by the report and generated PDF/figures; no recomputed partitions.

The recorded run used Python 3.12.14; matplotlib 3.10.8; numpy 2.3.5; reportlab 4.4.9; scikit-learn 1.8.0; scipy 1.17.0. The Java version and operating system are also saved in run_manifest.json. Input order and

random seeds are fixed. A different software environment can still affect floating-point details and execution time. [D11]

```
Selection SHA-256: 4b3328c198aeb11fd03a071231c707e1ab170e587f1d58f7a376ed49fc944854
Configuration SHA-256: d13a902f38da8e0e921eec0d5ae152ea4a8cac84d8de91546240b865bf07c069
```

Version and pilot ZIPs are stored as .zip.b64 text; the endpoint Git-source bundle remains one ZIP. The same unpack command decodes the text and verifies the stored file, decoded ZIP and every member before restoring the normal paths. Encoding changes storage only; all input and scientific file bytes are preserved. Tables, code and guides remain directly readable. See archives/README.md.

Adapting Java names for ACDC

ACDC expects file names and removes the final dotted suffix when it forms a basename. Passing Java class names directly could therefore collapse distinct classes to the same package name. The adapter maps p.A to p.A.java before running ACDC and reverses the mapping before evaluation. Every edge is preserved, and node_id_map.csv records the mapping. The original ACDC source is unchanged. [D7]

Checking the calculations independently

The complete test suite passed 91 tests covering extraction, class merging, Usage Equality, clustering, metrics, sampling and restoration. This includes 11 release-selection tests and 20 archive tests. The larger count below refers to individual assertions over the saved evidence, not to millions of separate unit tests. [D9; D13]

The independent audit passed: it performed 7,949,858 checks with 0 failures. A separate implementation, using Python's standard library, recalculates Usage Equality with exact fractions, lecture MQ and directional MoJoFM from the saved files. It checks all 120 final partitions, the 14 pilot inertia values and the AI output. These checks show that the calculations agree; they cannot establish that static extraction finds every dependency or that a recovered grouping matches the intended design. [D13]

Appendix A. The 30 selected releases

A.1 Results for all thirty releases

This table contains the counts behind the growth and filtering figures. Classes are project-owned types after merging inner and anonymous classes; edges are distinct directed dependencies. The filtering columns show the number of removed classes and the dependencies remaining afterwards. [D1; D4]

Release	Published	Classes	Edges	UE out	Edges left	Skipped
10.0	2011-09-28	317	1,289	6	1,036	—
11.0	2011-12-18	355	1,469	9	1,129	1
12.0	2012-04-30	373	1,597	8	1,257	2
13.0.1	2012-08-28	382	1,639	8	1,292	2
14.0	2013-02-25	412	1,945	12	1,384	0
15.0	2013-09-06	441	2,079	11	1,507	1
16.0	2014-01-17	448	2,121	10	1,544	0
17.0	2014-04-22	451	2,129	10	1,549	1
18.0	2014-08-25	454	2,112	10	1,535	0
19.0	2015-12-09	454	2,113	13	1,461	0
20.0	2016-10-28	514	2,453	18	1,589	0
21.0	2017-01-12	521	2,509	16	1,692	0
22.0	2017-05-22	529	2,534	16	1,712	0
23.0	2017-08-04	541	2,580	17	1,717	0
24.0	2018-02-01	549	2,633	17	1,771	6
25.0	2018-04-25	554	2,680	17	1,808	1
26.0	2018-08-01	556	2,686	17	1,812	3
27.0	2018-10-18	557	2,687	18	1,777	0
27.1	2019-03-08	556	2,692	18	1,784	1
28.0	2019-06-12	554	2,699	19	1,786	0
28.2	2019-12-27	558	2,727	19	1,810	1
29.0	2020-04-13	563	2,738	20	1,796	0
30.0	2020-10-16	569	2,769	19	1,899	0
31.0	2021-09-24	602	2,801	21	1,888	2
31.1	2022-02-28	603	2,811	22	1,829	1
32.0.0	2023-05-29	607	2,821	22	1,838	0
33.0.0	2023-12-18	605	2,826	23	1,830	5
33.4.0	2024-12-16	614	2,829	23	1,766	5
33.5.0	2025-09-17	590	2,744	21	1,732	8
33.7.2	2026-09-29	594	2,750	22	1,724	3

“UE out” counts removed classes. “Skipped” counts numeric stable tags strictly between this sample and the previous one, including any mainline or backport release excluded by the selection rules. The exact tags, Git commit identifiers and artifact variants are recorded in config/versions.json. [D4]

Appendix B. All final quality measurements

B.1 Releases 10.0 to 17.0

Selected releases 1–8 of 30. Each row gives the final result for one release and condition. The saved files include full precision and the individual cluster and cluster-pair terms. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
10.0	A1	317	55	0.2777	0.0044	0.2733	57%	35
10.0	A2	311	65	0.2173	0.0029	0.2144	52%	20
10.0	A3	317	23	0.1756	0.0070	0.1686	58%	46
10.0	A4	311	23	0.1981	0.0075	0.1905	52%	63
11.0	A1	355	64	0.2746	0.0043	0.2703	56%	35
11.0	A2	346	74	0.2383	0.0027	0.2356	49%	24
11.0	A3	355	26	0.1810	0.0064	0.1745	61%	42
11.0	A4	346	26	0.1850	0.0059	0.1791	50%	79
12.0	A1	373	70	0.2783	0.0055	0.2728	61%	29
12.0	A2	365	79	0.2523	0.0033	0.2490	53%	24
12.0	A3	373	27	0.1650	0.0074	0.1576	63%	36
12.0	A4	365	27	0.1828	0.0037	0.1791	50%	59
13.0.1	A1	382	80	0.2739	0.0047	0.2692	64%	36
13.0.1	A2	374	89	0.2397	0.0030	0.2367	58%	29
13.0.1	A3	382	28	0.1707	0.0083	0.1624	69%	53
13.0.1	A4	374	28	0.1907	0.0060	0.1847	56%	72
14.0	A1	412	92	0.3181	0.0040	0.3140	64%	57
14.0	A2	400	100	0.2927	0.0025	0.2902	56%	48
14.0	A3	412	30	0.1802	0.0068	0.1735	68%	48
14.0	A4	400	30	0.1886	0.0044	0.1841	54%	69
15.0	A1	441	93	0.3111	0.0041	0.3070	64%	41
15.0	A2	430	100	0.2976	0.0027	0.2949	55%	29
15.0	A3	441	32	0.1784	0.0065	0.1720	67%	51
15.0	A4	430	32	0.1647	0.0048	0.1598	54%	72
16.0	A1	448	95	0.3141	0.0038	0.3104	64%	43
16.0	A2	438	101	0.2911	0.0026	0.2885	55%	30
16.0	A3	448	33	0.1504	0.0073	0.1430	70%	51
16.0	A4	438	33	0.1609	0.0042	0.1568	53%	62
17.0	A1	451	96	0.3172	0.0041	0.3131	64%	41
17.0	A2	441	104	0.2884	0.0028	0.2856	57%	29
17.0	A3	451	33	0.1697	0.0072	0.1624	67%	39
17.0	A4	441	33	0.1944	0.0033	0.1911	50%	81

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

B.2 Releases 18.0 to 25.0

Selected releases 9–16 of 30. Each row gives the final result for one release and condition. The saved files include full precision and the individual cluster and cluster-pair terms. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
18.0	A1	454	99	0.3116	0.0036	0.3080	63%	43
18.0	A2	444	106	0.3042	0.0024	0.3019	55%	40
18.0	A3	454	33	0.1573	0.0070	0.1502	71%	56
18.0	A4	444	33	0.1629	0.0056	0.1573	59%	84
19.0	A1	454	97	0.3340	0.0037	0.3303	63%	54
19.0	A2	441	105	0.2938	0.0023	0.2915	54%	31
19.0	A3	454	33	0.1665	0.0073	0.1592	71%	55
19.0	A4	441	33	0.1982	0.0040	0.1943	53%	83
20.0	A1	514	103	0.3367	0.0039	0.3328	63%	50
20.0	A2	496	111	0.2802	0.0017	0.2785	49%	38
20.0	A3	514	38	0.1615	0.0058	0.1557	67%	58
20.0	A4	496	38	0.1825	0.0035	0.1790	54%	86
21.0	A1	521	102	0.3185	0.0033	0.3151	64%	50
21.0	A2	505	113	0.2579	0.0019	0.2561	52%	39
21.0	A3	521	38	0.1735	0.0071	0.1664	71%	57
21.0	A4	505	38	0.1786	0.0037	0.1749	50%	79
22.0	A1	529	102	0.3158	0.0032	0.3126	64%	44
22.0	A2	513	115	0.2588	0.0018	0.2570	52%	34
22.0	A3	529	39	0.1828	0.0056	0.1773	69%	60
22.0	A4	513	39	0.1997	0.0031	0.1966	50%	88
23.0	A1	541	105	0.3026	0.0033	0.2993	64%	51
23.0	A2	524	118	0.2721	0.0018	0.2703	54%	29
23.0	A3	541	40	0.1726	0.0065	0.1661	72%	60
23.0	A4	524	40	0.1747	0.0042	0.1705	55%	98
24.0	A1	549	110	0.3169	0.0037	0.3132	64%	55
24.0	A2	532	119	0.2894	0.0019	0.2875	54%	46
24.0	A3	549	40	0.1715	0.0067	0.1648	71%	63
24.0	A4	532	40	0.1755	0.0046	0.1709	59%	97
25.0	A1	554	110	0.3237	0.0035	0.3202	65%	49
25.0	A2	537	120	0.2806	0.0020	0.2786	55%	35
25.0	A3	554	40	0.1649	0.0060	0.1589	71%	69
25.0	A4	537	40	0.1928	0.0034	0.1895	53%	110

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

B.3 Releases 26.0 to 31.0

Selected releases 17–24 of 30. Each row gives the final result for one release and condition. The saved files include full precision and the individual cluster and cluster-pair terms. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
26.0	A1	556	110	0.3125	0.0036	0.3089	65%	48
26.0	A2	539	120	0.2852	0.0021	0.2831	55%	43
26.0	A3	556	41	0.1751	0.0069	0.1682	71%	65
26.0	A4	539	41	0.1963	0.0035	0.1928	55%	101
27.0	A1	557	110	0.3308	0.0035	0.3273	64%	53
27.0	A2	539	120	0.2840	0.0021	0.2819	55%	44
27.0	A3	557	41	0.1753	0.0057	0.1696	68%	56
27.0	A4	539	41	0.1807	0.0034	0.1773	54%	92
27.1	A1	556	110	0.3260	0.0034	0.3226	64%	55
27.1	A2	538	120	0.2849	0.0020	0.2829	54%	47
27.1	A3	556	41	0.1754	0.0066	0.1688	71%	62
27.1	A4	538	41	0.1918	0.0036	0.1883	51%	106
28.0	A1	554	111	0.3127	0.0035	0.3092	65%	49
28.0	A2	535	121	0.2822	0.0021	0.2801	55%	34
28.0	A3	554	40	0.1419	0.0055	0.1364	68%	57
28.0	A4	535	40	0.1809	0.0043	0.1766	58%	96
28.2	A1	558	112	0.3297	0.0035	0.3262	65%	48
28.2	A2	539	121	0.2802	0.0019	0.2782	55%	34
28.2	A3	558	41	0.1532	0.0071	0.1461	73%	60
28.2	A4	539	41	0.1848	0.0040	0.1808	57%	89
29.0	A1	563	117	0.3247	0.0035	0.3212	65%	49
29.0	A2	543	126	0.2965	0.0018	0.2947	56%	32
29.0	A3	563	41	0.1704	0.0054	0.1650	68%	74
29.0	A4	543	41	0.1669	0.0034	0.1635	57%	99
30.0	A1	569	118	0.3174	0.0036	0.3139	65%	47
30.0	A2	550	127	0.3011	0.0023	0.2988	58%	33
30.0	A3	569	42	0.1624	0.0053	0.1571	71%	70
30.0	A4	550	42	0.2136	0.0038	0.2098	55%	83
31.0	A1	602	149	0.2704	0.0022	0.2683	65%	45
31.0	A2	581	156	0.2376	0.0014	0.2362	57%	27
31.0	A3	602	44	0.1759	0.0058	0.1701	70%	91
31.0	A4	581	44	0.1784	0.0037	0.1747	60%	119

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

B.4 Releases 31.1 to 33.7.2

Selected releases 25–30 of 30. Each row gives the final result for one release and condition. The saved files include full precision and the individual cluster and cluster-pair terms. [D1; D9]

Release	Run	N	k	Cohesion	Coupling	MQ	Inter %	Largest
31.1	A1	603	147	0.2642	0.0023	0.2619	65%	49
31.1	A2	581	155	0.2463	0.0015	0.2449	55%	31
31.1	A3	603	44	0.1657	0.0064	0.1593	72%	95
31.1	A4	581	44	0.2132	0.0034	0.2098	53%	133
32.0.0	A1	607	149	0.2541	0.0021	0.2520	64%	49
32.0.0	A2	585	157	0.2348	0.0013	0.2335	55%	31
32.0.0	A3	607	44	0.1728	0.0057	0.1671	70%	95
32.0.0	A4	585	44	0.1693	0.0044	0.1650	62%	119
33.0.0	A1	605	147	0.2461	0.0020	0.2441	65%	48
33.0.0	A2	582	157	0.2334	0.0013	0.2321	55%	32
33.0.0	A3	605	44	0.1742	0.0047	0.1694	69%	100
33.0.0	A4	582	44	0.1974	0.0038	0.1936	58%	114
33.4.0	A1	614	154	0.2291	0.0018	0.2273	65%	47
33.4.0	A2	591	166	0.2126	0.0010	0.2116	53%	45
33.4.0	A3	614	45	0.1729	0.0059	0.1670	71%	106
33.4.0	A4	591	45	0.1912	0.0034	0.1878	55%	126
33.5.0	A1	590	134	0.2687	0.0023	0.2664	63%	53
33.5.0	A2	569	149	0.2379	0.0014	0.2365	54%	32
33.5.0	A3	590	43	0.1871	0.0060	0.1811	70%	95
33.5.0	A4	569	43	0.1911	0.0027	0.1884	51%	116
33.7.2	A1	594	135	0.2575	0.0022	0.2553	64%	46
33.7.2	A2	572	149	0.2431	0.0014	0.2417	53%	35
33.7.2	A3	594	43	0.1852	0.0062	0.1790	66%	85
33.7.2	A4	572	43	0.1883	0.0035	0.1848	51%	118

N: evaluated nodes. k: nonempty clusters. Inter %: percentage of graph edges that cross clusters. Largest: maximum cluster size. A1=ACDC/full; A2=ACDC/UE filtered; A3=KMeans/full; A4=KMeans/UE filtered. Rounded displayed terms can differ in their last digit from subtracting rounded cohesion and coupling.

References

Page numbers below refer to PDF pages unless a printed slide number is stated. Repository paths are relative to the project root.

Supplied teaching material

[AS] EPL484, Fall 2026, Project 1 instructions, 5 PDF pages. Eligibility p.1; data pp.2–3; AI and report pp.3–5.

[TUT] Project 1 tutorial, 12 PDF pages. Deadline p.2; relation types p.6; ACDC pp.7–8; clustering pp.10–12.

[L12] Lectures 1–2: Introduction, 50 PDF pages. Evolution, maintenance and S/P/E classification pp.15–33; Lehman's laws and limits pp.35–48.

[L3] Lecture 3: Requirements, Part 1, 38 PDF pages. Measurable definitions/automation pp.9–19; relationship and traceability models pp.27–33.

[L4] Lecture 4: Requirements, Part 2, 33 PDF pages. Traceability and its limits pp.11–15; feedback evidence pp.22–31.

[L5] Lecture 5: Architecture Evolution, 34 PDF pages. PDF p.9 = printed slide10; after the missing slide6, PDF and printed numbers differ by one.

[L67] Lectures 6–7: Architecture Reconstruction, 50 PDF pages. MQ equations and worked example pp.15–17; ACDC pp.19–24; older Noise method pp.46–47.

Research and primary documentation

[UE] Yano and Matsuo (2020), Moderate Detection and Removal of Omnipresent Modules in Software Clustering, ICSME, pp.662–666. Equations 1–3: p.663; procedure: p.664. https://doi.org/10.1109/ICSME46990.2020.00068

[NOISE] Constantinou, Kakarontzas and Stamelos (2015), Journal of Systems and Software 107, pp.142–157. The earlier significance-based method. https://doi.org/10.1016/j.jss.2015.05.065

[ACDC] Original ACDC author archive, preserved and SHA-256 pinned in third_party/acdc. https://www.eecs.yorku.ca/~bil/downloads/acdc.tar

[SK] scikit-learn 1.8 KMeans API and normalize API; actual installed version is recorded by the run. https://scikit-learn.org/1.8/modules/generated/sklearn.cluster.KMeans.html

[JVMS] Java Virtual Machine Specification 21, §5.4.3.2–5.4.3.4: field and method resolution. https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-5.html

[G1] Guava source repository and project overview. https://github.com/google/guava

[G2] Guava ReleaseHistory, with linked historical release notes. https://github.com/google/guava/wiki/ReleaseHistory

Repository evidence cited in this report

[D1] results/sizes.csv; metrics.csv; versions/*/*_metrics.json

[D2] docs/omnipresent.md; src/evolution/omnipresent.py; results/versions/*/ue.json

[D3] results/same_graph_comparisons.csv; versions/*/*_projected_to_retained_metrics.json

[D4] docs/project_selection.md; config/versions.json; data/provenance/tag_inventory.json; project_eligibility.json

[D5] docs/data_acquisition.md; scripts/fetch_data.py; data/provenance/acquisition.json and source/Git audits

[D6] docs/modeling_decisions.md; extractor source; data/graphs/*/provenance.json and diagnostics

[D7] docs/course_traceability.md; src/evolution/clustering.py; config/analysis.json; A1–A4 run metadata

[D8] results/pilot/pilot.json and associated pilot assignments

[D9] src/evolution/metrics.py; tests; results/versions/*/*_quality/

[D10] results/evolution.csv; canonical class and edge manifests

[D11] results/runtimes.csv; run_manifest.json; performance/ue_benchmark.json

[D12] results/ai/experiment.json and its exact prompt/input/output/quality artifacts

[D13] scripts/validate_results.py; results/validation.json, independent artifact audit

[D14] docs/ue_case_study.md; results/performance/case_checks.json, exact-fraction source-backed UE cases

[D15] scripts/audit_change_points.py; results/change_points.json, source and graph-difference audit

[D16] docs/mojofm.md; src/evolution/mojofm.py; saved directional partition agreement tables